

MATLAB-Empfänger von LRPT-Signalen des Meteor-M2-Satelliten

angefertigt am Fachgebiet

Kommunikation und Informationstheorie

Fakultät IV - Elektrotechnik und Informatik

Institut für Hochfrequenz und Halbleitersystemtechnologien

Autoren:	Anton Valentin Dilg	(466011)
	Ramin Leon Neymeyer	(466437)
	Ramon Rennert	(466893)
	Steffen August Sigwart	(505215)
Betreuer:	Dr.-Ing. Andreas Benzin	
Abgabedatum:	20. März 2026	

Zusammenfassung

In dieser Arbeit wird die Implementierung eines vollständigen digitalen Empfängers für das Low-Rate Picture Transmission (LRPT)-Signal des meteorologischen Satelliten Meteor-M2 vorgestellt. LRPT ersetzt das ältere analoge Automatic Picture Transmission-Verfahren und ermöglicht durch digitale Quadrature Phase-Shift Keying (QPSK)-Modulation, paketbasierte Übertragung nach Consultative Committee for Space Data Systems (CCSDS)-Standards sowie leistungsfähige Fehlerkorrektur eine höhere Bildauflösung und Robustheit gegenüber Störungen.

Ausgehend vom komplexen Basisbandsignal wird die gesamte digitale Signalverarbeitung in MATLAB realisiert. Dazu gehören Trägerfrequenz- und Taktrückgewinnung, QPSK-Demodulation, die Erzeugung von Hard- und Softbits sowie die anschließende Kanalkodierung. Die Fehlerkorrektur erfolgt gemäß CCSDS-Standard durch Faltungscodierung mit Viterbi-Dekodierung und Reed–Solomon-Blockcodes. Zur Rahmensynchronisation werden Synchronisationsmarker verwendet. Die senderseitige Pseudo-Randomisierung wird vor der Reed–Solomon-Dekodierung rückgängig gemacht.

Nach erfolgreicher Dekodierung werden die Daten gemäß der CCSDS-Protokollstruktur in VCDUs und M-PDUs rekonstruiert und anschließend zu Bildpaketen zusammengesetzt. Die JPEG-kodierten Bilddaten werden schließlich dekodiert und zu einzelnen Spektralkanälen sowie zu einem RGB-Kompositbild verarbeitet. Die ausgewertete Aufnahme liefert 3523 Bildpakete und zeitlich zugeordnete Kanalbilder mit jeweils 1568×680 Pixeln. Fehlende oder nicht dekodierbare Blöcke werden als Platzhalter dargestellt und durch Gültigkeitsmasken gekennzeichnet.

Abstract

This report presents the design and implementation of a complete digital receiver for the Low-Rate Picture Transmission (LRPT) signal of the Meteor-M2 weather satellite. LRPT replaces the legacy analog Automatic Picture Transmission system and provides higher image resolution and improved robustness by using Quadrature Phase-Shift Keying (QPSK) modulation, packet-based transmission compliant with Consultative Committee for Space Data Systems (CCSDS) standards, and advanced forward error correction.

Starting from the complex baseband signal, the entire digital signal processing chain is implemented in MATLAB. This includes carrier frequency and symbol timing recovery, QPSK demodulation, generation of hard and soft decision bits, and subsequent channel decoding. Error correction is performed according to the CCSDS standard using convolutional coding with Viterbi decoding, combined with Reed–Solomon block codes. Frame synchronization uses attached sync markers. The transmitter-side pseudo-randomization is reversed before Reed–Solomon decoding.

After successful decoding, the received data are reconstructed into CCSDS-compliant VCDUs and M-PDUs, which are then assembled into image packets. The JPEG-encoded image data are decoded to obtain individual spectral channels as well as a combined RGB image. The evaluated recording yields 3523 image packets and temporally aligned channel images of 1568×680 pixels each. Missing or undecodable blocks are represented by placeholders and identified by validity masks.

Inhaltsverzeichnis

1	Einleitung	1
2	Theoretischer Hintergrund	2
2.1	Modulation	3
2.2	Trägerfrequenzrückgewinnung	4
2.3	Taktrückgewinnung	9
2.4	Hard- und Softbits	14
2.5	Faltungscodierung	14
2.6	Viterbi-Algorithmus	15
2.7	Synchronisation und Framing	16
2.8	Pseudo-Randomisierung	19
2.9	Datenframe	20
2.10	JPEG	23
3	Implementation	25
3.1	Demodulation	26
3.2	Dekodierung	28
3.3	JPEG	31
3.4	Ergebnisse und Validierung	34
4	Fazit	36
	Literaturverzeichnis	37

Akronyme

APT	Automatic Picture Transmission
ASM	Attached Sync Marker
CADU	Channel Access Data Unit
CCSDS	Consultative Committee for Space Data Systems
CFO	Carrier Frequency Offset
CVCDU	Coded Virtual Channel Data Unit
DCT	Diskrete Kosinustransformation
FFT	Fast-Fourier-Transform
FHP	First Header Pointer
ISI	Inter-Symbol-Interferenz
LO	lokale Oszillator
LFSR	Linear Feedback Shift Register
LRPT	Low-Rate Picture Transmission
MCU	Minimum Coded Unit
M-PDU	Multiplexing Protocol Data Unit
NOAA	National Oceanic and Atmospheric Administration
OQPSK	Offset Quadrature Phase-Shift Keying
OSI	Open Systems Interconnection
PCA-PDU	Physical Channel Access Protocol Data Unit
PLL	Phase-locked loop
PP	Partial Packet
QPSK	Quadrature Phase-Shift Keying
SDR	Software Defined Radio
VCDU	Virtual Channel Data Unit
VCO	Voltage Controlled Oscillator
VCID	Virtual Channel Identifier

Abbildungsverzeichnis

2.1	Überblick der digitalen Signalverarbeitung.	2
2.2	LRPT-Protokoll.	3
2.3	Konstellationsdiagramme von QPSK.	4
2.4	4th-Power-Methode zur CFO-Schätzung.	6
2.5	Schematische Darstellung einer Costas Loop für QPSK.	8
2.6	Impulsantwort der Puls-Autokorrelationsfunktion (Raised Cosine).	10
2.7	Schematische Darstellung eines Mueller-und-Müller-Detektors.	11
2.8	Darstellung eines Symbolübergangs mit Abtastwerten e , d , c und b	12
2.9	Schematische Darstellung eines Gardner-Detektors [10].	13
2.10	Schematische Darstellung eines Faltungscodierers.	15
2.11	Trellis-Diagramm.	17
2.12	Rotationen und Symmetrie der Konstellation.	18
2.13	Pseudo-Randomisierer Logik Diagramm.	19
2.14	Spektralkoeffizienten der zweidimensionalen DCTs-II mit $N_1 = N_2 = 8$	24
2.15	Zig-Zag Sequenz der quantisierten DCTs Koeffizienten.	24
3.1	Frequenzdarstellung des empfangenen Signals.	26
3.2	Konstellationsdiagramme des Signals vor und nach der Demodulation.	27
3.3	Normierter Betrag der Kreuzkorrelation des faltungscodierten ASM mit einem LLR-Block. Die Peaks liegen im Abstand von 16 384 Kanalbits.	28
3.4	Betrag der Kreuzkorrelation des ASM 1ACFFC1D mit dem Viterbi-dekodierten Bitstrom.	29
3.5	Vollständiger 24-Bit-VCDU-Zähler über die 925 ausgewerteten Frames.	30
3.6	Dekodierte Bildkanäle und RGB-Komposit mit jeweils 1568×680 Pixeln. Graue Platzhalter in den Einzelkanälen kennzeichnen nicht rekonstruierte Bildblöcke.	33

Tabellenverzeichnis

2.1	Aufbau einer VCDU.	20
2.2	Aufbau eines Bildpakets.	22
3.1	Ergebnisse für die untersuchte I/Q-Aufnahme.	35

1 Einleitung

Die älteren National Oceanic and Atmospheric Administration (NOAA)-Satelliten nutzten das analoge Automatic Picture Transmission (APT) Verfahren. Mit der Meteor-M-Serie wurde dieses durch das digitale Low-Rate Picture Transmission (LRPT) ersetzt [1], [2]. Die Umstellung bringt mehrere Vorteile:

- **Effiziente Spektrumnutzung:** Beide Systeme nutzen denselben Frequenzbereich um 137 MHz, jedoch erlaubt die digitale Quadrature Phase-Shift Keying (QPSK)-Modulation eine deutlich höhere Informationsdichte pro Hertz. Dadurch können bei gleicher Bandbreite mehr Bilddaten übertragen werden [3].
- **Höhere Bildauflösung:** APT liefert meist 4 km/Pixel [4], während LRPT eine Auflösung von etwa 1 km/Pixel ermöglicht [2]. Dies erlaubt detailliertere Erd- und Wolkenaufnahmen.
- **Fehlerkorrektur:** APT-Bilder verschlechtern sich kontinuierlich mit abnehmendem Signal-Rausch-Verhältnis. Bei LRPT wird Vorwärtsfehlerkorrektur eingesetzt, wodurch viele Bitfehler korrigiert werden können [5].
- **Paketbasierte Datenübertragung:** LRPT folgt den Consultative Committee for Space Data Systems (CCSDS)-Standards und kapselt die Informationen in Datenpakete. Dies erlaubt die gleichzeitige Übertragung von Bilddaten und Telemetrie und entspricht dem Open Systems Interconnection (OSI)-Schichtenmodell [5].
- **Kompatibilität:** Während APT spezielle analoge Empfänger erforderte, kann LRPT mit günstigen Software Defined Radios (SDR) (z. B. RTL-SDR) und frei verfügbarer Open-Source-Software (z. B. SatDump) dekodiert werden. Die Daten liegen direkt digital vor und können leicht archiviert und weiterverarbeitet werden.

Im Rahmen dieses Projekts wird ein Empfänger für die Meteor-M-Satellitenserie in MATLAB implementiert, der die komplette Dekodierung und Signalverarbeitung übernimmt. Der vollständige Quellcode inklusive Beispieldaten ist unter der MIT-Lizenz öffentlich auf GitHub verfügbar.

2 Theoretischer Hintergrund

Um das empfangene Signal und den anschließenden Dekodierprozess nachvollziehen zu können, werden in diesem Abschnitt die relevanten theoretischen Grundlagen erläutert. Abbildung 2.1 zeigt den prinzipiellen Aufbau der gesamten Verarbeitungskette. Ausgehend vom niederfrequenten I/Q-Signal wird das empfangene QPSK-Signal zunächst demoduliert, wobei Frequenz-, Phasen- und Taktsynchronisation sukzessive hergestellt werden. Darauf folgt die Synchronisation und Rahmenerkennung, in der die empfangenen Symbole in strukturierte Datenblöcke überführt und mittels Viterbi-Decodierung sowie Derandomisierung in einen fehlerreduzierten Bitstrom konvertiert werden. Abschließend erfolgt die Dekodierung der äußeren Codes sowie die Extraktion der eigentlichen Nutzdaten (grün), welche schließlich zu JPEG-Bilddaten zusammengesetzt werden.

Während das Blockdiagramm die physikalische Verarbeitungskette beschreibt, verdeutlicht Abbildung 2.2 die logische Struktur der Datenrahmen gemäß dem CCSDS-Standard 131.0-B-5. Jede Schicht kapselt die darunterliegende vollständig, wodurch eine hierarchische Datenorganisation entsteht, von der physikalischen Übertragungsebene mit Physical Channel Access Protocol Data Unit (PCA-PDU) über Channel Access Data Unit (CADU) sowie Virtual Channel Data Unit (VCDU) bis hin zu den Multiplexing- und Applikationspaketen, Multiplexing Protocol Data Unit (M-PDU) sowie Minimum Coded Unit (MCU). Diese Struktur bildet die Grundlage der späteren Decodierung und erlaubt eine fehlerrobuste und standardisierte Übertragung der Telemetriedaten.

Im Folgenden werden die einzelnen Blöcke und Protokollschichten im Detail beschrieben, beginnend bei der Demodulation und Synchronisation bis hin zur Fehlerkorrektur und der Extraktion der Bildinformationen.

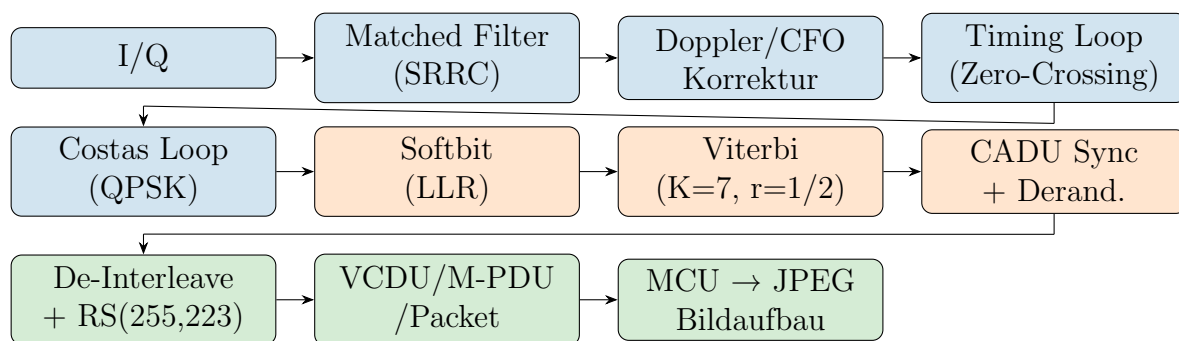


Abbildung 2.1: Blockdiagramm der digitalen Signalverarbeitung im Meteor-Empfänger. Das empfangene QPSK-Signal wird schrittweise demoduliert (blau), synchronisiert (orange) und dekodiert (grün).

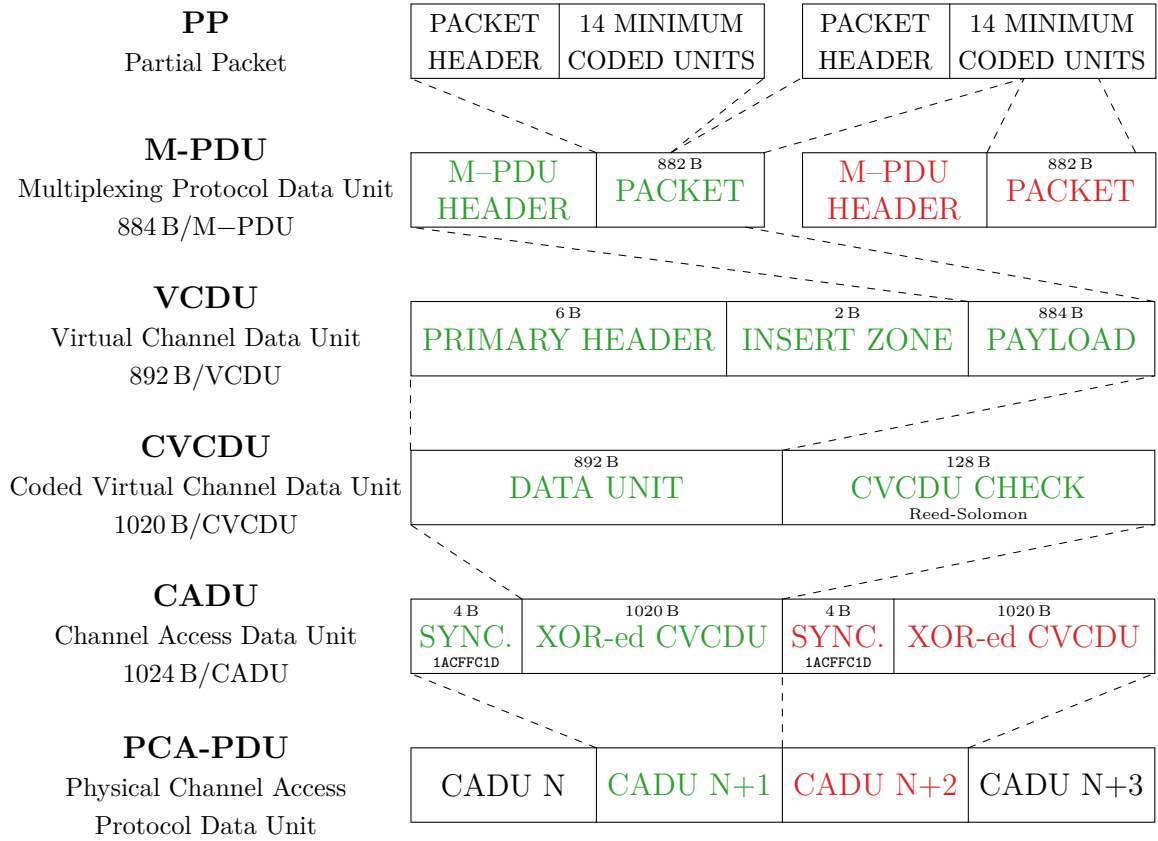


Abbildung 2.2: Protokollschichten, welche durchlaufen werden, um das Signal in ein Bild umzuwandeln.

2.1 Modulation

Das LRPT-Sendesignal des Meteor-M2 benutzt QPSK. Normale QPSK-Modulation codiert jeweils zwei Bits pro Symbol, indem der Träger in eine von vier Phasenlagen

$$\varphi \in \{\pm 45^\circ, \pm 135^\circ\} \quad (2.1)$$

bzw. in Radiand

$$\varphi \in \left\{ \pm \frac{\pi}{4}, \pm \frac{3\pi}{4} \right\} \quad (2.2)$$

versetzt wird. Im Basisband kann dies als Folge komplexer Symbole

$$a_k \in \{+1 + j, +1 - j, -1 + j, -1 - j\}$$

beschrieben werden, wobei ein Gray-Mapping verwendet wird, damit Nachbarpunkte sich nur in einem Bit unterscheiden. Ein Nachteil darin ist, dass Phasensprünge von bis zu 180° auftreten können, etwa beim Übergang von $+1 + j$ zu $-1 - j$. Solche Sprünge führen zu starken Hüllkurvenschwankungen, die zu Verzerrungen führen [6]. Im Konstellationsdiagramm 2.3 werden diese vier Punkte dargestellt.

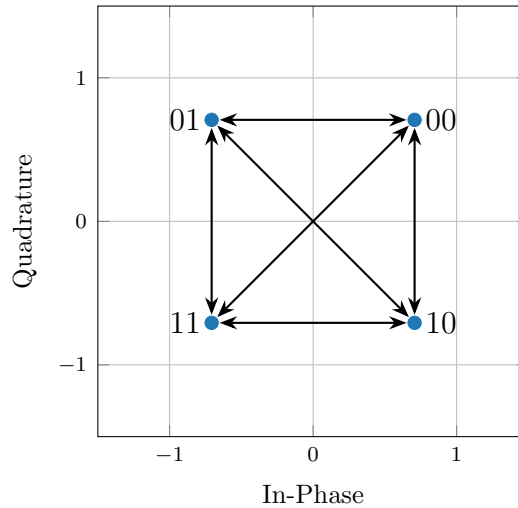


Abbildung 2.3: Konstellationsdiagramme von QPSK.

2.2 Trägerfrequenzrückgewinnung

Ein zentrales Problem bei der Demodulation von QPSK- und Offset Quadrature Phase-Shift Keying (OQPSK)-Signalen ist der Carrier Frequency Offset (CFO). Dieser tritt auf, wenn die Trägerfrequenz des Senders und der lokale Oszillator (LO) des Empfängers nicht exakt übereinstimmen. Ursachen können Oszillatordrift durch Temperatur- und Alterungseffekte, Abweichungen in der Abtastrate sowie der Doppler-Effekt bei Satellitenübertragungen sein.

Das empfangene Signal $r(t)$ kann mit einem zusätzlichen Frequenzoffset f_{offset} modelliert werden als

$$r(t) = s(t) \cdot e^{j2\pi f_{\text{offset}} t}, \quad (2.3)$$

wobei $s(t)$ das ideal empfangene Basisbandsignal bezeichnet. Der Faktor $e^{j2\pi f_{\text{offset}} t}$ entspricht einer kontinuierlichen Rotation der IQ-Konstellation. Im Zeitbereich äußert sich CFO als Schwebung, im Konstellationsdiagramm laufen die Symbolpunkte kreisförmig um den Ursprung. Die CFO-Kompensation erfolgt typischerweise in zwei Schritten. Zu-

nächst wird eine Grobschätzung im Open-Loop-Verfahren durchgeführt und später eine Feinregelung in einer Closed-Loop [7].

4th-Power-Methode Zunächst zum Open-Loop-Verfahren: Befindet sich im Basisband ein signifikanter Frequenzoffset, verschiebt sich das Trägersignal im Spektrum. Eine Möglichkeit zur Schätzung besteht darin, eine Fourier-Transformation des Empfangssignals durchzuführen und den größten Spektralpeak zu identifizieren. Da das Signal jedoch QPSK-moduliert ist, ist im Frequenzbereich oft kein eindeutiger Peak sichtbar. Hier setzt die sogenannte M -th-Power-Methode an. Dabei wird das Signal mit der Ordnung M potenziert, wobei M der Anzahl der Konstellationspunkte entspricht. Durch diesen Schritt überlagern sich die Phasen aller Symbole, sodass die Modulation eliminiert wird und alle Punkte in der Konstellation übereinanderfallen [8]. Für eine komplexe Zahl z in Polarform und ihre vierten Potenz gilt

$$|z^4| = |z|^4, \quad \angle z^4 = 4 \cdot \angle z. \quad (2.4)$$

Die Phasen der vier Punkte einer regulären QPSK-Konstellation 2.2 werden durch Multiplikation dieser Phasen mit 4 alle Punkte auf denselben Ort abgebildet. Da

$$+\pi = -\pi = +3\pi = -3\pi, \quad (2.5)$$

gilt auch

$$4 \cdot \left\{ \pm \frac{\pi}{4}, \pm \frac{3\pi}{4} \right\} = \pi. \quad (2.6)$$

Diese Phasenmultiplikation kann für QPSK-Symbole $a[m]$ geschrieben werden als

$$4 \cdot \angle r[m] = \pi, \quad (2.7)$$

unabhängig davon, welchen Wert jedes Symbol $r[m]$ hat. Die zugrundeliegende Intuition ist die $\frac{2\pi}{4}$ -Rotationssymmetrie der QPSK-Konstellation. Eine Drehung der Konstellation um $\frac{2\pi}{4}$ erzeugt dieselbe Konstellation. Dies kann implementiert werden, indem man $a[m]$ zur vierten Potenz erhebt und ist in Abbildung 2.4a dargestellt. Nun den Fall, dass die QPSK-Symbole am Empfänger zusätzlich einen Phasenoffset θ_Δ enthalten. Die Datensymbole werden durch $r[m]$ bezeichnet, deren In-Phase- und Quadraturanteile $r_I[m]$ bzw. $r_Q[m]$ sind. Das Ausgangssignal des Matched Filters am Empfänger, heruntergetastet auf ein Sample pro Symbol, sei $z(mT_M)$, wobei m der Symbolindex und T_M die

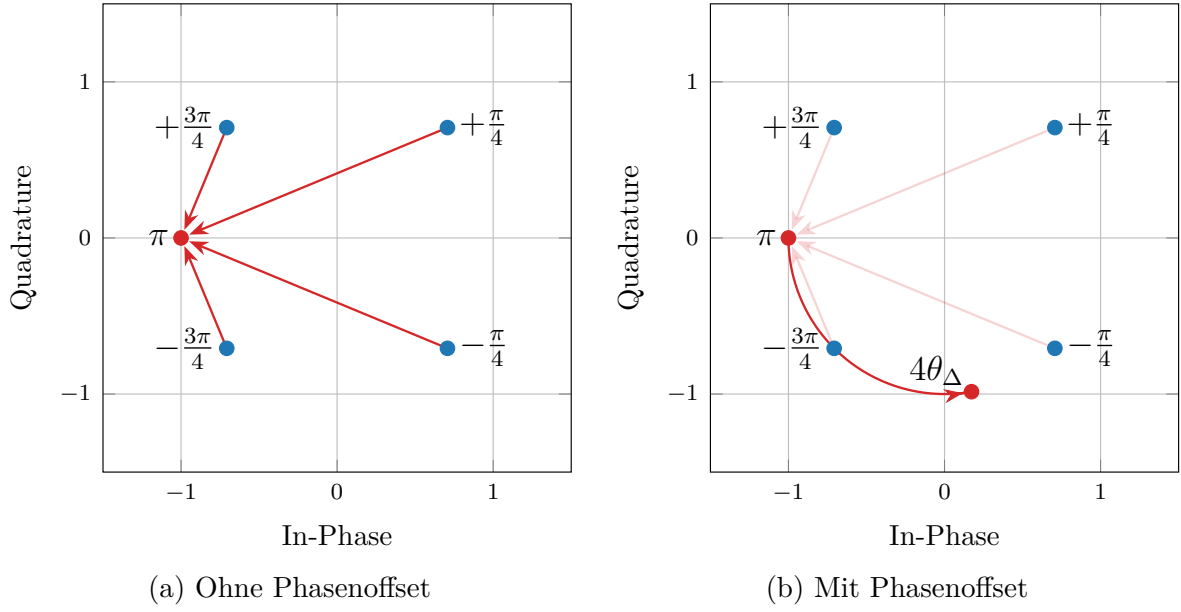


Abbildung 2.4: 4th-Power-Methode zur CFO-Schätzung.

Symboldauer ist. Zudem gilt aus [7], dass Betrag und Phase des Matched-Filter-Ausgangs zum Symbolzeitpunkt m geschrieben werden können als

$$|z(mT_M)| = \sqrt{r_I^2[m] + r_Q^2[m]}, \quad \angle z(mT_M) = \angle r[m] + \theta_\Delta. \quad (2.8)$$

Ein Mismatch bzw. Offset des Winkels θ_Δ zwischen dem eingehenden Träger und dem Oszillator des Empfängers verändert also nicht den Betrag der Datensymbole $a[m]$, sondern rotiert lediglich ihre Phasen um θ_Δ . Wird nun jedes Ausgangssample zur vierten Potenz erhoben, so ergibt sich

$$|z^4(mT_M)| = \left(r_I^2[m] + r_Q^2[m]\right)^2, \quad \angle z^4(mT_M) = 4 \cdot \angle r[m] + 4\theta_\Delta = \pi + 4\theta_\Delta. \quad (2.9)$$

Dies ist in Abbildung 2.4b dargestellt. In dieser Gleichung kann die Summation der Matched-Filter-Ausgänge genutzt werden, um das Rauschen zu mitteln und ein grober Phasenschätzer kann abgeleitet werden als

$$\hat{\theta}_\Delta = \frac{1}{4} \cdot \angle \left\{ \sum_{m=0}^{N_d-1} z^4(mT_M) \right\} - \frac{\pi}{4}, \quad (2.10)$$

wobei N_d die Sequenzlänge in Symbolen ist. In der Fast-Fourier-Transform (FFT) des potenzierten Signals r^M bleibt dann lediglich eine Sinuswelle bei der Frequenz $M f_{\text{offset}}$

bestehen. Der Frequenzoffset kann somit als

$$f_{\Delta} = \frac{f_{\text{peak}}}{M} \quad (2.11)$$

abgelesen werden. Mit dieser Schätzung lässt sich das empfangene Signal durch Multiplikation mit einem Korrektorexponential entsprechend dem negativen Offsetfrequenz korrigieren

$$r_{\text{corr}}[n] = r[n] \cdot e^{-j2\pi f_{\Delta} n}. \quad (2.12)$$

Costa Loop Nach der groben Korrektur wird die verbleibende Abweichung in Frequenz und Phase durch einen geschlossenen Regelkreis, der sogenannten Costas Loop, kompensiert [7], welcher eine spezielle Form einer Phasenregelung, einer Phase-locked loop (PLL), darstellt [9]. Die Grundidee besteht darin, das empfangene Signal gleichzeitig mit einem lokal erzeugten Sinus- und Cosinussignal zu multiplizieren. Dadurch entstehen ein In-Phase-Zweig (I) und ein Quadratur-Zweig (Q), die anschließend durch Tiefpassfilter von hochfrequenten Anteilen befreit werden. Liegt ein Frequenz- oder Phasenfehler vor, zeigt sich dieser als Abweichung in der Energieverteilung zwischen den beiden Zweigen. Aus I- und Q-Signal wird ein Fehlersignal gebildet, das proportional zur Phasendifferenz zwischen empfangenem Träger und lokalem Oszillator ist. Dieses Fehlersignal wird durch einen Loop-Filter geglättet und steuert den LO, in diesem Fall ein Voltage Controlled Oscillator (VCO), der Frequenz und Phase kontinuierlich nachführt. Der Regelkreis arbeitet solange, bis die Phasendifferenz $\theta - \phi$ verschwindet und das Signal auf der In-Phase-Achse ausgerichtet ist. Auf diese Weise korrigiert der Costas Loop nicht nur Restfrequenzoffsets, sondern fixiert auch die Phasenlage der Konstellation, sodass eine stabile Symbolerkennung ermöglicht wird [10]. Um die Funktionsweise genauer zu verstehen, wird mit dem Empfangssignal $r(t)$ begonnen, das die Trägerwelle mit einem Phasenoffset θ_{Δ} enthält und einen positiven Vorfaktor beim Quadraturträger aufweist:

$$r(t) = v_I(t)\sqrt{2}\cos(2\pi F_c t + \theta_{\Delta}) + v_Q(t)\sqrt{2}\sin(2\pi F_c t + \theta_{\Delta}). \quad (2.13)$$

Am Empfänger wird das Signal in die I- und Q-Zweige aufgeteilt und jeweils mit den Sinus-/Cosinus-Signalen eines VCOs multipliziert, dessen Phase auf $\hat{\theta}_{\Delta}$ eingestellt ist. Für den I-Zweig ergibt sich:

$$x_I(t) = r(t) \cdot \sqrt{2}\cos(2\pi F_c t + \hat{\theta}_{\Delta}). \quad (2.14)$$

Mit trigonometrischen Identitäten [11] und nach Tiefpassfilterung bleibt übrig:

$$z_I(t) = v_I(t) \cos \theta_{\Delta e} + v_Q(t) \sin \theta_{\Delta e}, \quad (2.15)$$

wobei $\theta_{\Delta e} = \theta_{\Delta} - \hat{\theta}_{\Delta}$ der Phasenfehler ist. Analog ergibt sich für den Q-Zweig:

$$x_Q(t) = r(t) \cdot \sqrt{2} \sin(2\pi F_c t + \hat{\theta}_{\Delta}) \quad (2.16)$$

und nach Anwendung der trigonometrischen Identitäten und Tiefpassfilterung

$$z_Q(t) = -v_I(t) \sin \theta_{\Delta e} + v_Q(t) \cos \theta_{\Delta e}. \quad (2.17)$$

Unter der Annahme einer rechteckigen Pulsform und eines kleinen Phasenfehlers (gilt näherungsweise $\cos \theta_{\Delta e} \approx 1$, $\sin \theta_{\Delta e} \approx 0$) [8], folgt

$$z_I(t) \approx v_I(t), \quad z_Q(t) \approx v_Q(t). \quad (2.18)$$

Die Vorzeichenerkennung dieser Signale liefert somit eine Schätzung der übertragenen Symbole:

$$\text{sign}\{z_I(t)\} \approx \hat{a}_I[m], \quad \text{sign}\{z_Q(t)\} \approx \hat{a}_Q[m]. \quad (2.19)$$

Das Ziel dieser Vorzeichenoperationen ist es, kleine Schwankungen, die durch das Signal

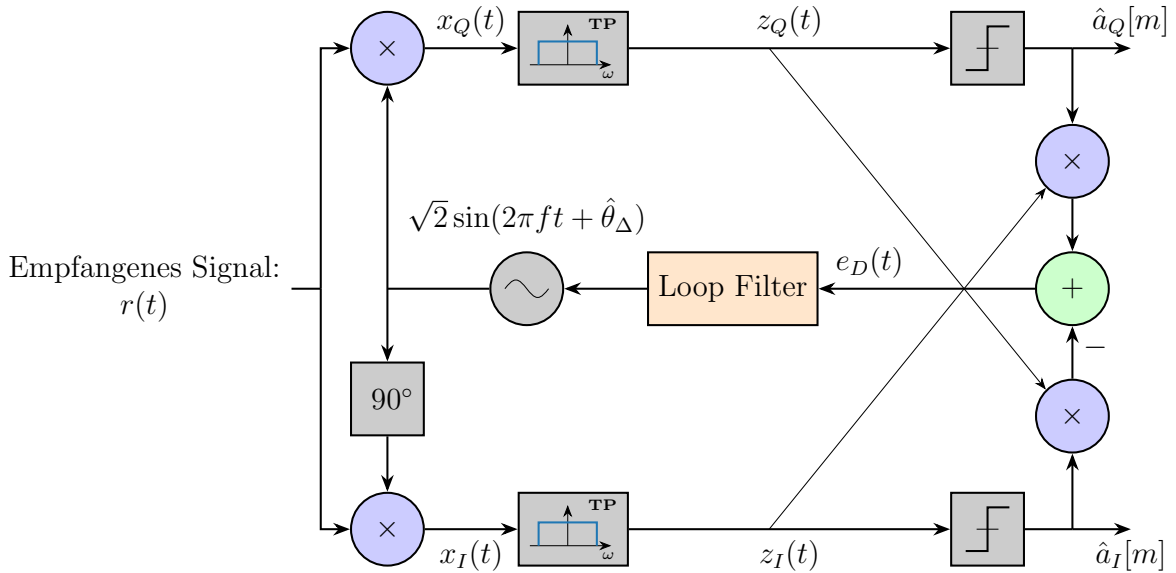


Abbildung 2.5: Schematische Darstellung einer Costas Loop für QPSK.

im jeweils anderen Zweig oder durch Rauschen verursacht werden, auszublenden [12]. Der Fehlersignal-Ausdruck des Costas-Loops ergibt sich daher zu

$$e_D(t) = \text{sign}\{z_Q(t)\}z_I(t) - \text{sign}\{z_I(t)\}z_Q(t). \quad (2.20)$$

Dies kann für kleine Phasenfehler näherungsweise als

$$e_D[m] = \hat{a}_I[m]z_Q(t) - \hat{a}_Q[m]z_I(t) \quad (2.21)$$

geschrieben werden. Dieses Fehlersignal treibt den Phasenregelkreis an, bis der Phasenfehler verschwindet und die Symbole stabil auf den Achsen der QPSK-Konstellation liegen.

2.3 Taktrückgewinnung

Die präzise Symbol-Taktsynchronisation ist eine zentrale Voraussetzung für die digitale Demodulation, da schon kleine Abweichungen im Abtastzeitpunkt zu Inter-Symbol-Interferenz (ISI) führen und die Symbolentscheidungen verfälschen können. Zur kontinuierlichen Nachführung des optimalen Abtastzeitpunkts werden daher spezielle Timing-Fehlerdetektoren eingesetzt. Im Folgenden werden zwei verbreitete Vertreter vorgestellt: der Mueller-und-Müller-Algorithmus [13] sowie der Gardner-Detektor [9].

Mueller-und-Müller-Algorithmus Der Algorithmus wird anhand einer Impulsantwort einer Puls-Autokorrelationsfunktion, am Beispiel eines Raised-Cosine-Pulses erklärt, welche in Abbildung 2.6 dargestellt ist. Wenn man nahezu im Optimum abtastet, also $\varepsilon_\Delta \approx 0$, dann sind die beiden Nachbar-Samples, die jeweils ein Symbol vor und nach dem Maximum bei $\pm T_M$ liegen, fast gleich groß. Dies liegt daran, dass nahe des Maximums der Autokorrelation alle anderen symbolabständigen Abtastungen in der Nähe der Nulldurchgänge liegen [14]. Aus dieser Beobachtung scheint es, als müssten drei Abtastwerte verwendet werden, um einen Ausdruck für einen Timing-Fehlerdetektor zu konstruieren. Ein Sample für den Symbolwert selbst, um die Modulation zu entfernen und zwei weitere für die Bestimmung der Steigung. Allerdings benötigt man mit dem Algorithmus von nur zwei benachbarten Symbolabständen. Es gilt die Beziehung [15]

$$r_p(+T_M - \varepsilon_\Delta) \approx r_p(-T_M - \varepsilon_\Delta) \quad (2.22)$$

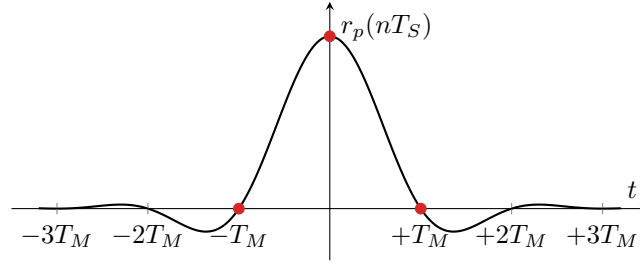


Abbildung 2.6: Impulsantwort der Puls-Autokorrelationsfunktion (Raised Cosine).

für $\varepsilon_\Delta \approx 0$. Sobald der Timing-Fehler größer wird, entsteht eine Asymmetrie. Wenn ε_Δ negativ ist, wird das rechte Sample größer als das linke. Die Differenz der beiden Terme liefert daher ein Fehlersignal, das proportional zum tatsächlichen Timing-Offset ε_Δ ist. Dieses Signal kann in einen Synchronisations-Loop eingespeist werden.

Um die beiden Terme $r_p(+T_M - \varepsilon_\Delta)$ und $r_p(-T_M - \varepsilon_\Delta)$ zu gewinnen, nutzt man die Symbole $a[m-1]$ und $a[m]$: Das Matched-Filter-Sample $z(mT_M)$ enthält auch einen Beitrag vom Symbol $m-1$, der genau $r_p(+T_M - \varepsilon_\Delta)$ entspricht. Multipliziert man $z(mT_M)$ mit $a[m-1]$, wird die Modulation entfernt und dieser Term isoliert. Analog liefert das Sample $z((m-1)T_M)$ zusammen mit $a[m]$ den Anteil $r_p(-T_M - \varepsilon_\Delta)$. Daraus kann man dann den Error Term herleiten [16]. Das ideale Sendesignal kann geschrieben werden als

$$s(nT_S) = \sum_i a[i] p(nT_S - iT_M), \quad (2.23)$$

wobei $a[i]$ die Modulationssymbole und $p(nT_S)$ die Square-Root-Nyquist-Pulsform ist. T_S und T_M bezeichnen die Abtast- bzw. Symboldauer. Wird das Signal mit einem Timing-Fehler ε_Δ empfangen, ergibt sich

$$r(nT_S) = \sum_i a[i] p(nT_S - iT_M - \varepsilon_\Delta). \quad (2.24)$$

Am Empfänger ergibt sich nach der Matched-Filterung

$$z(nT_S) = r(nT_S) * p(-nT_S) = \left\{ \sum_i a[i] p(nT_S - iT_M - \varepsilon_\Delta) \right\} * p(-nT_S) \quad (2.25)$$

$$= \sum_i a[i] r_p(nT_S - iT_M - \varepsilon_\Delta), \quad (2.26)$$

wobei $r_p(\cdot)$ die Puls-Autokorrelationsfunktion ist. Für eine anfängliche Timing Schätzung

$\hat{\varepsilon}_\Delta = 0$ und Abtastung mit $L = 1$ Sample pro Symbol gilt zum Zeitpunkt mT_M :

$$z(mT_M) = \sum_i a[i] r_p(mT_M - iT_M - \varepsilon_\Delta) = \sum_i a[i] r_p((m-i)T_M - \varepsilon_\Delta). \quad (2.27)$$

Für binäre PAM-Symbole $a[i] \in \{-1, +1\}$ gilt:

$$\text{Mean}\{a[m] \cdot a[i]\} = \begin{cases} 1, & i = m, \\ 0, & \text{sonst.} \end{cases} \quad (2.28)$$

Damit folgt

$$\text{Mean}\{a[m-1] \cdot z(mT_M)\} = r_p(+T_M - \varepsilon_\Delta), \quad (2.29)$$

und analog

$$\text{Mean}\{a[m] \cdot z((m-1)T_M)\} = r_p(-T_M - \varepsilon_\Delta). \quad (2.30)$$

Vergleicht man beide Ausdrücke mit der Symmetriebedingung aus Gleichung 2.22, so ergibt sich

$$r_p(+T_M - \varepsilon_\Delta) - r_p(-T_M - \varepsilon_\Delta) = a[m-1] \cdot z(mT_M) - a[m] \cdot z((m-1)T_M). \quad (2.31)$$

So kann ein Timing Fehlerdetektor konstruiert werden als

$$e_D[m] = a[m-1] z(mT_M + \hat{\varepsilon}_\Delta) - a[m] z((m-1)T_M + \hat{\varepsilon}_\Delta). \quad (2.32)$$

Dies ist die data-aided Version des Mueller-und-Müller-Detektors. Für die decision-directed Variante werden die Symbole $a[m]$ durch ihre Entscheidungen $\hat{a}[m]$ ersetzt

$$e_D[m] = \hat{a}[m-1] z(mT_M + \hat{\varepsilon}_\Delta) - \hat{a}[m] z((m-1)T_M + \hat{\varepsilon}_\Delta). \quad (2.33)$$

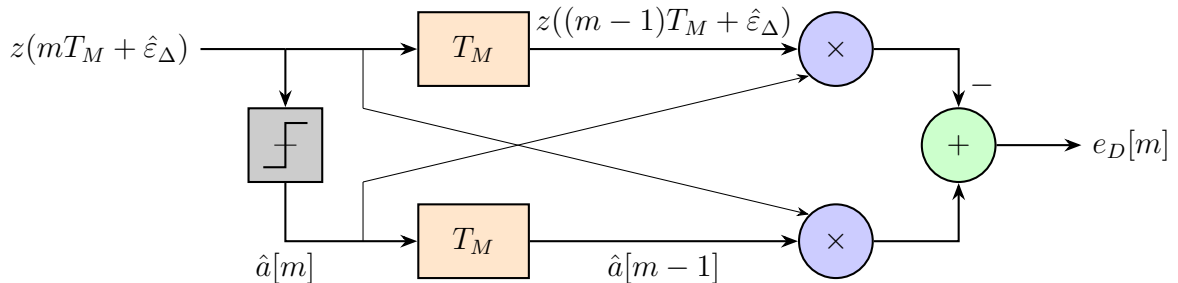


Abbildung 2.7: Schematische Darstellung eines Mueller-und-Müller-Detektors.

Gardner-Algorithmus Nun zum Gardner-Algorithmus. Dieser ist non-data-aided und benötigt daher keine bekannten Trainingssymbole. Stattdessen nutzt er zusätzlich zum Symbolsample ein Mid-Symbol-Sample bei halber Symboldauer.

Betrachtet man die Matched-Filter-Ausgabe $z(nT_S)$, so ergibt sich eine Sequenz von Abtastwerten des gefilterten Empfangssignals. Um den Symbolübergang von $a[m-1]$ nach $a[m]$ zu verstehen, wählt man drei charakteristische Samples wie in Abbildung 2.8 dargestellt.

Diese Abtastwerte werden im Folgenden sinngemäß als e (linkes Sample zum Zeitpunkt $(m-1)T_M$), d (Mid-Symbol-Sample bei $mT_M - T_M/2$) und c (rechtes Sample bei mT_M) bezeichnet. Liegt der Abtastzeitpunkt exakt im Optimum, so befindet sich die mittlere Probe d im Mittel nahe dem Nulldurchgang der Puls-Autokorrelationsfunktion (Zero Crossing), während die Randproben e und c die beiden Symbolwerte repräsentieren. Ein Timing-Offset verschiebt dagegen die Lage von d , was vom Gardner-Detektor zur Bestimmung des Fehlersignals ausgenutzt wird.

Ein klassischer Early-Late-TED vergleicht die Steigung mithilfe der Differenz zweier Randproben und gewichtet sie mit der mittleren Probe. Je nach Dreierauswahl (b, c, d) bzw. (c, d, e) in der Skizze kann das Vorzeichen des resultierenden Fehlers aber uneindeutig werden. Einmal $d \cdot (c - e) > 0$ und einmal $d \cdot (c - e) < 0$, was einen Zero-Crossing-Ansatz motiviert [16].

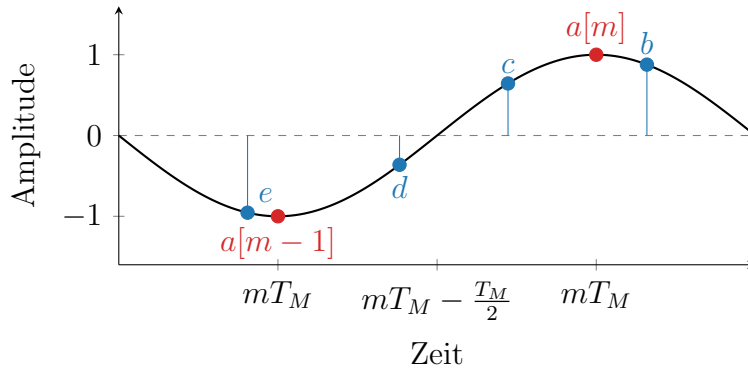


Abbildung 2.8: Darstellung eines Symbolübergangs mit Abtastwerten e , d , c und b .

Die Zero-Crossing-Strategie negiert die Steigung in der Early-Late-Formel, sodass die Regelgröße den Abtastzeitpunkt so verschiebt, dass die mittlere Probe gegen Null konvergiert.

Also der Fehler sieht wie folgt aus [15]:

$$e_D[m] = z \left(mT_M - \frac{T_M}{2} + \hat{\varepsilon}_\Delta \right) \left\{ z((m-1)T_M + \hat{\varepsilon}_\Delta) - z(mT_M + \hat{\varepsilon}_\Delta) \right\}. \quad (2.34)$$

Intuitiv heißt das, dass wenn man zu früh abtastet d im Mittel das gleiche Vorzeichen wie die Differenz der Randproben hat. Wird zu früh abgetastet, hat d (die Mitte) im $e_D[m]$ positiv ist, also der Loop schiebt nach „später“. Ist zu spät abgetastet, kehrt sich die Situation um, also $e_D[m] < 0$, und der Loop schiebt nach „früher“. Der Regler arbeitet so lange, bis die mittlere Probe gegen den Zero Crossing driftet und die Randproben die Symbolmitten treffen.

Setzt man nun anstelle der Randproben die Symbole ein, erhält man eine data-aided Form (Zero-Crossing TED) [9].

$$e_D[m] = z \left(mT_M - \frac{T_M}{2} + \hat{\varepsilon}_\Delta \right) \{ a[m-1] - a[m] \}. \quad (2.35)$$

Oft nutzt man die decision-directed Variante

$$e_D[m] = z \left(mT_M - \frac{T_M}{2} + \hat{\varepsilon}_\Delta \right) \{ \hat{a}[m-1] - \hat{a}[m] \}, \quad (2.36)$$

wobei für binäre PAM-Symbole mit Amplitude A

$$\hat{a}[m] = A \cdot \text{sign} \{ z(mT_M + \hat{\varepsilon}_\Delta) \}. \quad (2.37)$$

Für QPSK wendet man (2.34) komponentenweise auf die In-Phase- und Quadratur-Zweige an und summiert die beiden Fehlerbeiträge.

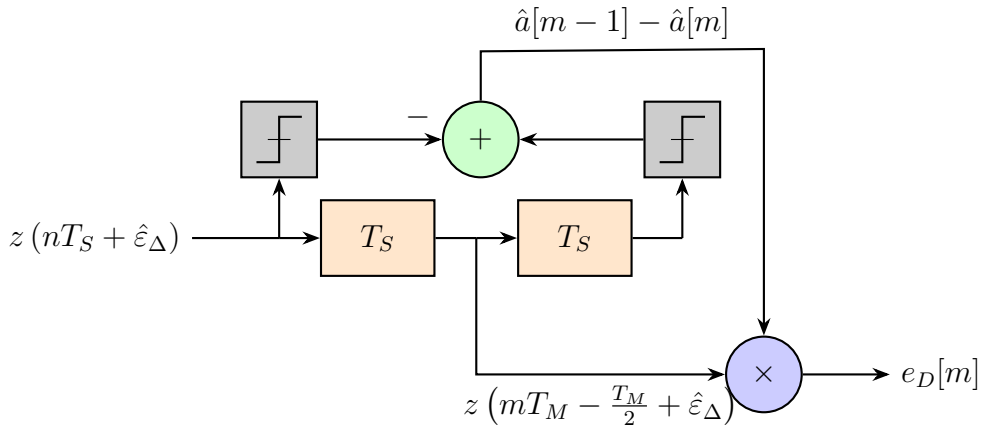


Abbildung 2.9: Schematische Darstellung eines Gardner-Detektors [10].

2.4 Hard- und Softbits

Nach der Demodulation entstehen zunächst Werte im zweidimensionalen Signalraum. Jedes Symbol trägt damit zwei Bitinformationen, die aus den Vorzeichen von I und Q bestimmt werden, wie in Abbildung 2.3 dargestellt.

Bei Hardbits erfolgt eine unmittelbare Zuordnung anhand der Entscheidungsgrenzen in der Konstellation. Liegt das empfangene Symbol beispielsweise im rechten oberen Quadranten, so werden die Bits direkt als „00“ interpretiert. Das Ergebnis ist ein reiner Bitstrom aus 0 und 1, in dem keinerlei Information über die Zuverlässigkeit der Entscheidung enthalten ist. Hard-Decision-Decoding ist einfach zu implementieren, verzichtet jedoch auf die statistische Information über die Entfernung des Symbols zur Entscheidungsgrenze.

Softbits hingegen berücksichtigen diese Distanz zur Entscheidungsgrenze. Anstatt die Entscheidung sofort hart auf 0 oder 1 zu quantisieren, wird für jedes empfangene Symbol ein Wert berechnet, der angibt, wie wahrscheinlich ein Bit „0“ oder „1“ ist. In dem Fall von QPSK entspricht dies typischerweise der Projektion des Empfangsvektors auf die I - bzw. Q -Achse: Ein Symbol mit $I = +1.8$ deutet auf ein sehr sicheres Bit „0“ in der I -Komponente hin, während $I = +0.1$ nur eine unsichere Entscheidung für „0“ liefert. Das gleiche gilt für die Q -Komponente [17].

2.5 Faltungscodierung

Die Faltungscodierung führt Redundanz in den Datenstrom ein, indem jedes Informationsbit nicht nur isoliert übertragen wird, sondern sich über mehrere nachfolgende Codierschritte auswirkt. Im Gegensatz zu Blockcodes erfolgt die Verarbeitung dabei taktweise, sodass keine Verzögerungen durch Blockbildung entstehen. Dies macht Faltungscodes sinnvoll für Echtzeitanwendungen [6]. Außerdem verarbeiten Faltungsdecoder verdrauschte Empfangssequenzen nicht nur als harte Bitfolgen, sondern die Zuverlässigkeitsinformation jedes Symbols direkt in den Entscheidungsprozess einbeziehen können. Während beim Hard-Decision-Decoding die Hamming-Distanz zwischen empfangenen und zulässigen Codewörtern herangezogen wird, nutzen Soft-Decision-Algorithmen den euklidischen Abstand im Signalraum. Dadurch sinkt die Restbitfehlerwahrscheinlichkeit erheblich, was insbesondere in Satellitenkanälen mit starkem Rauschen entscheidend ist [18].

Die Codierung selbst ist dabei ausgesprochen einfach, da sie nur aus Schieberegistern

und XOR-Operationen besteht.

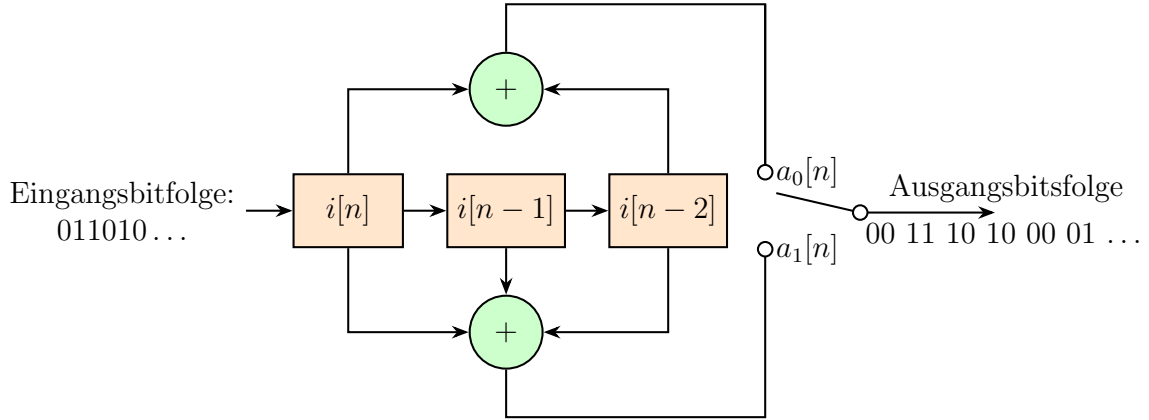


Abbildung 2.10: Schematische Darstellung eines Faltungscodierers mit der Coderate $r = 1/2$ und Constraint Length $K = 3$.

Ein Faltungscoder lässt sich als Schieberegister mit $K - 1$ Speicherstellen beschreiben, wobei K die sogenannte Constraint Length ist. Das aktuelle Eingangsbit u_k sowie die Inhalte des Schieberegisters werden mit Generatorpolynomen g_i verknüpft, wodurch mehrere Ausgänge v_i entstehen [19]:

$$v_i[k] = u_k \cdot g_i[0] \oplus_2 u_{k-1} \cdot g_i[1] \oplus_2 \dots \oplus_2 u_{k-(K-1)} \cdot g_i[K-1], \quad (2.38)$$

wobei $\oplus_2$ die Addition im endlichen Körper $GF(2)$ bezeichnet, die äquivalent zur logischen XOR-Verknüpfung ist [6]. Abbildung 2.10 zeigt beispielhaft einen Faltungscodierer der Rate $r = 1/2$ mit Constraint Length $K = 3$ und den zugehörigen XOR-Verknüpfungen. Im CCSDS-Standard wird ein Faltungscode mit $K = 7$ und Rate $r = 1/2$ verwendet unter Verwendung der Generatorpolynome $G_1 = 171_8$ und $G_2 = 133_8$ (im Oktalsystem), der nach demselben Prinzip aufgebaut ist [20].

2.6 Viterbi-Algorithmus

Die Codierung der Daten ist eigentlich relativ einfach. Die Herausforderung liegt im Decodierprozess, da hierfür aus allen möglichen Bitpfaden derjenige gefunden werden muss, der mit höchster Wahrscheinlichkeit gesendet wurde. Dafür wird der Viterbi-Algorithmus genutzt. Der Viterbi-Algorithmus ist ein Maximum-Likelihood-Sequenzschätzer [21]. Ein Faltungscode kann als Zustandsmaschine beschrieben werden, deren Zustände den Inhalt

des Schieberegisters repräsentieren. Jedes neue Eingangsbit bewirkt einen Übergang zu einem Folgezustand und erzeugt gleichzeitig eine bestimmte Folge von Ausgangsbits. Die Menge aller möglichen Zustandsübergänge lässt sich als Trellis-Diagramm darstellen. Der Viterbi-Algorithmus sucht in diesem Trellis den Pfad mit der maximalen Wahrscheinlichkeit, d. h. die Sequenz von Eingangsbits, die am besten zu den empfangenen, meist verrauschten Bits passt [17].

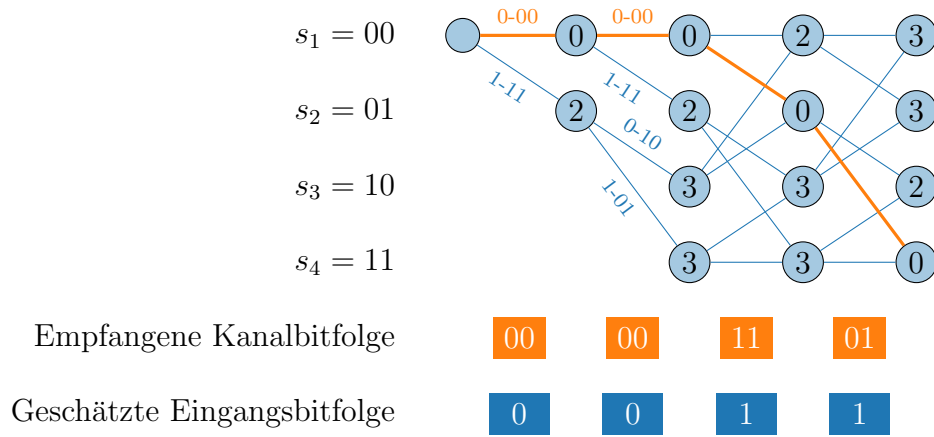
Für jedes Paar empfangener Kanalbits des Rate- $\frac{1}{2}$ -Codes werden alle möglichen Zustandsübergänge im Trellis betrachtet. Die Güte eines Übergangs wird über eine Pfadmetrik bewertet. Bei Hard-Decision-Decoding entspricht diese der Hamming-Distanz zwischen empfangenen und erwarteten Codebits. Bei Soft-Decision-Decoding entspricht sie dem euklidischen Abstand oder einer Likelihood-Metrik, wodurch auch die Zuverlässigkeit der empfangenen Softbits berücksichtigt wird. Zu jedem Zeitpunkt werden nur die wahrscheinlichsten Übergänge weiterverfolgt, die sogenannten Survivor Paths, wodurch die Komplexität reduziert wird. Am Ende wird der gesamte wahrscheinlichste Pfad durch Traceback rekonstruiert und die zugehörige Bitfolge ausgegeben. Bei einem Soft-Decision-Decoding ergibt sich ein deutlicher Leistungsgewinn gegenüber einfachen Hard-Decision-Ansätzen, weshalb dies auch in diesem Projekt genutzt wird [6].

In Abbildung 2.11 der Ablauf des Viterbi-Algorithmus dargestellt. Im oberen Diagramm ist der Fall einer fehlerfreien Übertragung gezeigt. Die empfangene Kanalbitfolge stimmt exakt mit einer gültigen Codesequenz überein, sodass der Decoder ohne Abweichungen dem richtigen Pfad folgt. Im unteren Diagramm tritt dagegen ein Fehler im zweiten Symbolpaar bzw. dritten Bit auf. Lokale Abweichungen entstehen, dennoch wählt der Algorithmus aufgrund der globalen Maximum-Likelihood-Betrachtung den korrekten Gesamtpfad. Somit wird trotz Übertragungsfehlern wieder die richtige Informationsbitfolge rekonstruiert [6].

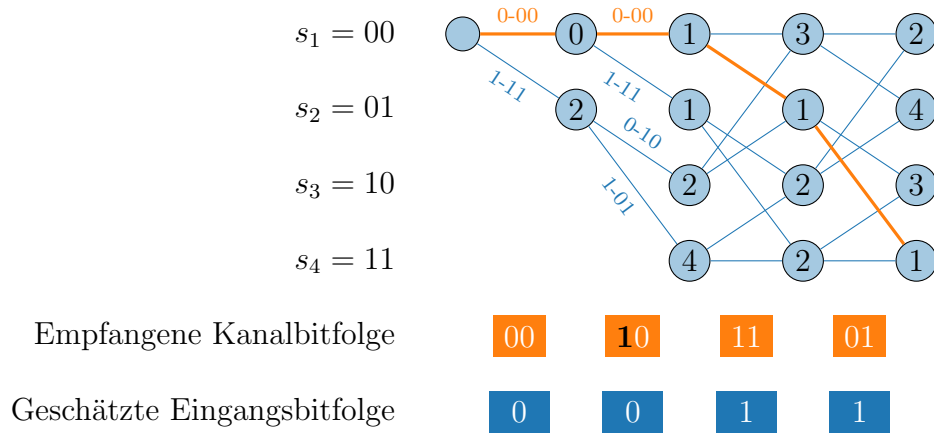
2.7 Synchronisation und Framing

Nach der symbol- und bitweisen Decodierung liegt am Ausgang des Viterbi-Decoders ein kontinuierlicher Bitstrom vor, der keine explizite Information über die Grenzen einzelner Datenrahmen enthält. Um die weiteren Protokollschichten korrekt zu rekonstruieren, muss dieser Strom zunächst segmentiert, synchronisiert und anschließend entrandomisiert werden.

Eine CADU stellt die physikalische Übertragungseinheit der Telemetriedaten dar. Jede



(a) Fehlerfreie Übertragung.



(b) Fehlerhafte Übertragung.

Abbildung 2.11: Rückverfolgung des wahrscheinlichsten Pfades im Trellis mit fehlerfreier sowie fehlerhafte Übertragung mit Fehlern im zweiten Symbolpaar.

besitzt eine feste Länge von 1024 B, bestehend aus einem 4 B-Synchronisationswort und einer 1020 B langen Nutzlast, wie in Abbildung 2.2 dargestellt. Das Synchronisationswort oder auch Attached Sync Marker (ASM) entspricht der hexadezimalen Sequenz 1ACFFC1D im dekodierten Bereich und FCA2B63DB00D9794 im kodierten Bereich, also vor dem Viterbi-Dekoder [20]. Es dient der eindeutigen Rahmenerkennung innerhalb des Bitstroms. Die Rahmensynchronisation erfolgt durch eine bitweise Korrelation des empfangenen Bitstroms mit dem bekannten Syncword. Bei Überschreiten einer definierten Korrelationsschwelle wird eine mögliche Framegrenze angenommen und nach Erreichen eines stabilen Locks wird der Bitstrom in Blöcke fester Länge von 1024 B segmentiert. Diese Blöcke bilden die einzelnen CADUs.

Konstellations Rotation Ein verbleibendes Problem betrifft die verschiedenen möglichen Symbolmappings innerhalb der QPSK-Konstellations. Die Phase kann hier vier mögliche Zustände annehmen, von denen jeder ein Bitpaar codiert. Die Wahl der Zuordnung zwischen Phasenlage und Bitkombination ist jedoch nicht eindeutig und schon eine falsche Symbol-Bit-Zuordnung führt zu einer völlig anderen Bitfolge, die nicht mehr mit der Referenzsequenz korreliert.

Eine Bitinversion lässt sich bei Hardbits durch Vertauschen von null und eins und bei LLRs durch einen Vorzeichenwechsel darstellen. Die Zuverlässigkeitsinformation der LLRs bleibt dabei erhalten. Eine Vertauschung der beiden Bitpositionen entspricht einer Umordnung der zugehörigen Softwerte. Alternativ werden die verschiedenen Symbolmappings als Operationen auf den Real- und Imaginärteilen (I, Q) der empfangenen Symbole beschrieben. Achsentausch und Vorzeichenwechsel erfassen die entsprechenden Rotationen und Spiegelungen.

So wird in Abbildung 2.12a der Imaginärteil $\Im\{\cdot\}$ invertiert, wodurch eine Spiegelung an der In-Phase-Achse (I-Achse) entsteht. Analog dazu zeigt Abbildung 2.12b den Fall, bei dem der Realteil $\Re\{\cdot\}$ negiert wird. Dies führt zu einer Spiegelung an der Q-Achse. In Abbildung 2.12c ist schließlich der Fall dargestellt, bei dem die beiden Achsen vertauscht werden, also $(I, Q) \mapsto (Q, I)$. Die letzte Variante, in Abbildung 2.12d gezeigt, kombiniert beide Negationen $(I, Q) \mapsto (-Q, -I)$. Zudem wird die zu Beginn vorgestellte Variante in Abbildung 2.3 mit allen dazugehörigen Rotationen genutzt.

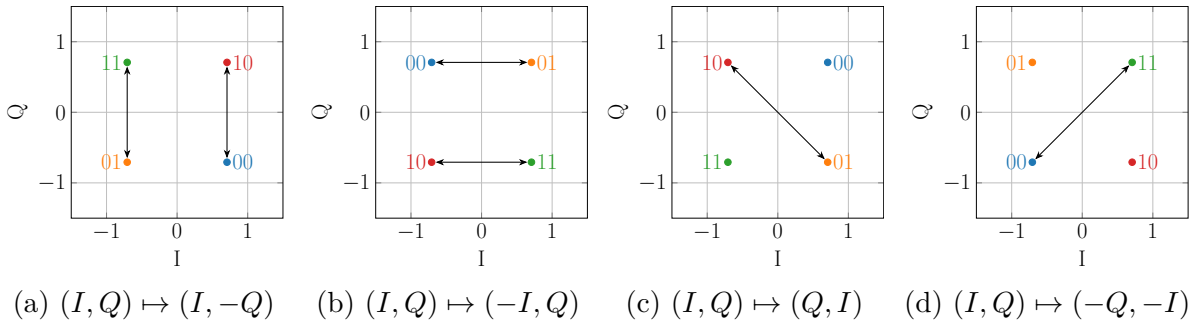


Abbildung 2.12: Rotationen und Symmetrie der Konstellation.

Nur eine eindeutige Zuordnung der Konstellationspunkte zu den entsprechenden Bitpaaren führt zu einer periodischen Abfolge von Korrelationspeaks mit dem Synchronisationswort FCA2B63DB00D9794 für faltungscodierte Signale [20]. Dies ist die richtige Zuordnung, die in den weiteren Dekodierungsschritten verwendet wird.

Die übertragenen Bits im Satelliten werden durch eine Pseudozufallssequenz mit XOR verknüpft, damit das Signal weitgehend zufällig aussieht, also wenig Periodizität und eine ausreichende Bit-Übergangsdichte aufweist. Die Randomisierung wird beim Sender vor der Faltungscodierung angewendet. Beim Empfänger erfolgt die Derandomisierung daher nach der Viterbi-Dekodierung und vor der Reed–Solomon-Korrektur. Das ASM, also das Synchronisationswort, wird nicht randomisiert und muss vor der Derandomisierung von den CADUs entfernt werden, da es sonst zu fehlerhaften Zuständen des Linear Feedback Shift Register (LFSR) kommen würde. Die Pseudozufallssequenz ist als maximale LFSR-Sequenz mit dem Polynom

definiert, wobei die Anfangszustände des LFSR auf 1 festgelegt wurden [20]. Diese Randomisierung wird am Empfänger rückgängig gemacht, indem dieselbe PN-Sequenz (Periodenlänge 255 Bits) lokal erzeugt und erneut XOR-verknüpft wird:

Der Aufbau eines solchen Pseudo-Randomisierer ist in Abbildung 2.13 dargestellt. Das Ergebnis ist die ursprüngliche Coded Virtual Channel Data Unit (CVCDU).



2.9 Datenframe

Nach der Reed–Solomon-Dekodierung werden die Datenblöcke zu VCDUs zusammengesetzt, welche in den folgenden Schritten weiter zu M-PDUs und schließlich zu Bildpaketen verarbeitet werden. Der Zweck der virtuellen Kanäle liegt in der logischen Trennung mehrerer Datenströme innerhalb einer physischen Übertragungsverbindung [22].

VCDU Eine VCDU besteht aus insgesamt 892 B und setzt sich aus drei Komponenten zusammen: einem 6 B großen Primärheader, einer 2 B umfassenden Insert Zone sowie einem Datenfeld von 884 B. Das Datenfeld enthält hier eine M-PDU mit zwei Headerbytes und einer 882 Byte langen Packet Zone. Der VCDU Header beinhaltet mehrere Felder, darunter:

- die Version Number (2 Bit) (für LRPT 01),
- die VCDU ID, bestehend aus der Spacecraft ID (8 Bit) und der Virtual Channel Identifier (VCID) (6 Bit),
- einen VCDU Counter (24 Bit)
- sowie ein Signalling Field (8 Bit), das ein Replay-Flag zur Unterscheidung von Echtzeit- und Wiederholungsdaten enthält.

Tabelle 2.1: Beispielhafter Aufbau einer VCDU mit zugehörigen Header-, Insert- und M-PDU-Feldern.

...	64	64	64	64	...	Version
...	5	5	5	5	...	Type
...	9	9	9	9	...	Counter
...	191	191	191	191	...	:
...	101	102	103	104	...	:
...	0	0	0	0	...	Signalling Field
...	0	0	0	0	...	VCDU Insert Zone
...	0	0	0	0	...	:
...	0	1	0	2	...	M-PDU Header
...	18	142	44	194	...	:
...	113	47	70	115	...	M-PDU Payload
...	:	:	:	:	...	:

Die Insert Zone wird für Verschlüsselungsinformationen genutzt, ist jedoch im Falle von LRPT-Übertragungen mit Nullen gefüllt, da keine Kryptographie angewendet wird. Das abschließende Data Unit Field (884 Byte) enthält die eigentlichen Nutzdaten in Form einer M-PDU [23]. Eine beispielhafte Struktur einer VCDU ist in Tabelle 2.1 dargestellt.

M-PDU Die M-PDU beginnt mit einem Header aus fünf Reservebits und einem 11 bit langen First Header Pointer (FHP). Es folgen 882 Bytes Packet Zone, die vollständige Pakete oder Paketfragmente enthalten können [24]. Der Pointer bezeichnet den nullbasierten Byteoffset des ersten neuen Paketheaders innerhalb dieser Zone. Für eine bei eins beginnende Indizierung gilt

$$\text{Byteposition} = P + 1. \quad (2.41)$$

Bytes vor dieser Position gehören gegebenenfalls zur Fortsetzung eines Pakets aus dem vorherigen Frame. Beginnt im Frame kein neuer Paketheader, wird der Pointer auf 2047 gesetzt. Auch ein am Frame-Ende beginnender Header kann im nächsten Frame fortgesetzt werden. Die Paketlänge und die Kontinuität der Frame-Zähler sind deshalb zusätzlich zum Pointer auszuwerten. Fülldaten werden nicht als Bildpakete interpretiert.

Partial Packets Ein Partial Packet (PP) besteht aus einem sechs Byte langen Primärheader, einem acht Byte langen Zeitstempel und den anschließenden Nutzdaten [24]. Für die hier verarbeiteten Bildpakete sind folgende Felder relevant:

- Paket-ID (16 Bit): drei Bits Paketversion, ein Typbit, ein Sekundärheader-Kennzeichen und elf Bits APID. Die APID ordnet das Paket einem Bildkanal zu.
- Sequenzsteuerung (16 Bit): zwei Sequenzflags und ein 14-Bit-Paketzähler für die jeweilige Quelle.
- Paketlängenfeld (16 Bit): Anzahl der Bytes nach dem Primärheader minus eins. Für den Feldwert L beträgt die gesamte Paketlänge daher $6 + L + 1$ Bytes.
- Zeitstempel (64 Bit): zwei Bytes Tagesnummer, vier Bytes Millisekunden des Tages und zwei Bytes Mikrosekunden. In der Meteor-M2-Spezifikation sind Tagesnummer und Mikrosekundenfeld mit null belegt; das Millisekundenfeld enthält den Aufnahmezeitpunkt des Bildstreifens.
- MCU-Startnummer (8 Bit): Position der ersten MCU des Pakets innerhalb des Bildstreifens.

-
- Scan-Header (16 Bit): Index der Quantisierungstabelle sowie die Indizes der Huffman-Tabellen für DC und AC. Die verwendeten Indizes sind null.
 - Segmentheader (24 Bit): Qualitätsfaktormarker FFF0 und der Qualitätsfaktor als ein Byte.
 - Quelldaten: 14 komprimierte MCUs mit den Bildinformationen.

Tabelle 2.2: Beispielhafter Aufbau eines Bildpakets (PP).

...	8	8	8	8	...	
...	68	68	64	64	...	APID
...	57	58	122	58	...	Sequenzsteuerung
...	255	0	2	3	...	:
...	0	0	0	0	...	Länge
...	125	57	175	97	...	:
...	0	0	0	0	...	Datum
...	0	0	0	0	...	:
...	2	2	2	2	...	:
...	149	149	149	149	...	:
...	131	131	135	135	...	:
...	8	8	216	216	...	:
...	0	0	0	0	...	Mikrosekunden
...	0	0	0	0	...	:
...	168	0	0	14	...	MCU Nummer
...	0	0	0	0	...	Scan-Header
...	0	0	0	0	...	:
...	255	255	255	255	...	Qualitätsfaktormarker
...	240	240	240	240	...	:
...	79	79	79	79	...	Qualitätsfaktor
...	:	:	:	:	...	14 MCUs

2.10 JPEG

Im Payload eines PPs sind jeweils 14 MCUs enthalten, die gemäß dem JPEG-Standard dekomprimiert werden müssen [25], [26]. Eine MCU beschreibt dabei Bildinformationen für ein 8×8 Pixel großes Teilbild. Die JPEG-Kodierung basiert auf der zweidimensionalen Diskrete Kosinustransformation (DCT), deren Basisfunktionen in Abbildung 2.14 dargestellt sind. Der links oben dargestellte Koeffizient entspricht dem DC-Anteil und repräsentiert den mittleren Helligkeitswert des Blocks, während die übrigen 63 Koeffizienten die AC-Anteile und damit unterschiedliche horizontale und vertikale Frequenzen beschreiben.

Jedes 8×8 -Teilbild kann als Linearkombination dieser Basisfunktionen dargestellt werden. Übertragen werden daher nicht die Pixelwerte selbst, sondern die zugehörigen DCT-Koeffizienten. Da das menschliche Auge gegenüber hochfrequenten Bildanteilen weniger empfindlich ist, werden diese Koeffizienten zunächst quantisiert, wobei insbesondere hohe Frequenzen stärker reduziert werden. Anschließend erfolgt eine Entropiekodierung mittels Huffman-Codierung [27], ergänzt durch eine Run-Length-Kodierung, um längere Folgen von Nullen effizient darzustellen.

Vor der Entropiekodierung werden die quantisierten DCT-Koeffizienten in eine eindimensionale Sequenz überführt. Dies geschieht mithilfe der in Abbildung 2.15 dargestellten Zig-Zag-Anordnung, die niedrige Frequenzen priorisiert und hochfrequente Anteile am Ende der Sequenz bündelt. Dadurch entstehen längere Nullsequenzen, was die Effizienz der Run-Length-Kodierung erhöht.

Eine Besonderheit stellt die Kodierung der DC-Koeffizienten dar: Diese werden nicht absolut, sondern als Differenz zum DC-Koeffizienten des vorhergehenden 8×8 -Blocks übertragen. Für den ersten Block jedes Bildpakets wird der DC-Prädiktor auf null gesetzt; die erste übertragene Differenz entspricht damit dem absoluten DC-Koeffizienten. Die konkrete Rekonstruktion der Koeffizienten und der anschließende Rücktransformationsprozess werden in Abschnitt 3 detailliert beschrieben.

Beim Empfang werden die komprimierten Daten entsprechend decodiert und die DCT-Koeffizienten rückgewonnen. Durch inverse Quantisierung und inverse DCT können schließlich Näherungen der ursprünglichen Helligkeitswerte der Bilddaten rekonstruiert werden [28]. Für die inverse Quantisierung wird der Qualitätsfaktor genutzt, welcher in Tabelle 2.2 z.B. den Wert 79 aufweist.

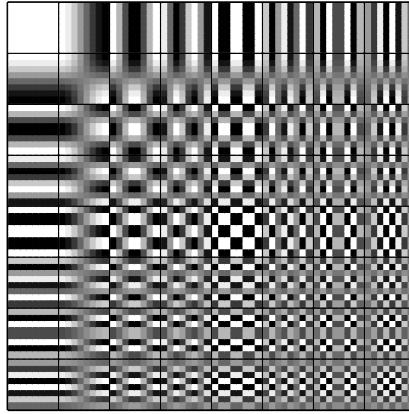


Abbildung 2.14: Spektralkoeffizienten der zweidimensionalen DCTs-II mit $N_1 = N_2 = 8$.

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

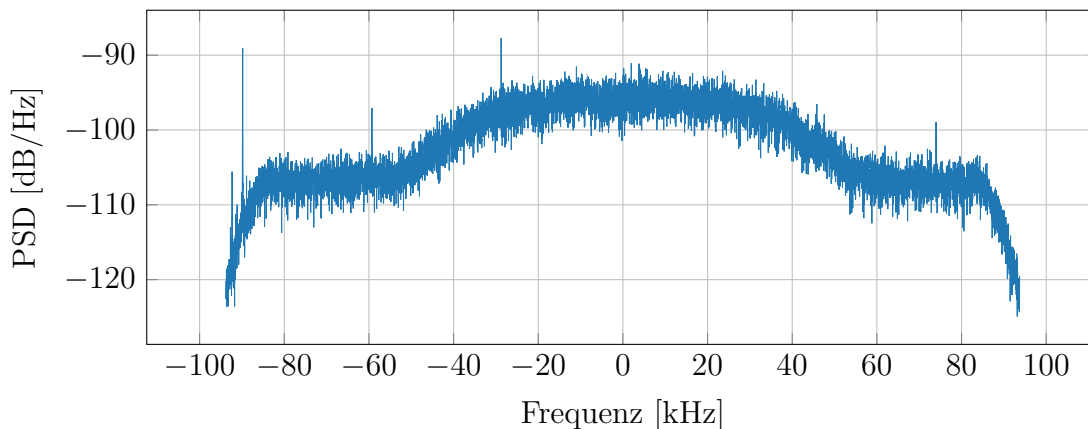
Abbildung 2.15: Zig-Zag Sequenz der quantisierten DCTs Koeffizienten.

3 Implementation

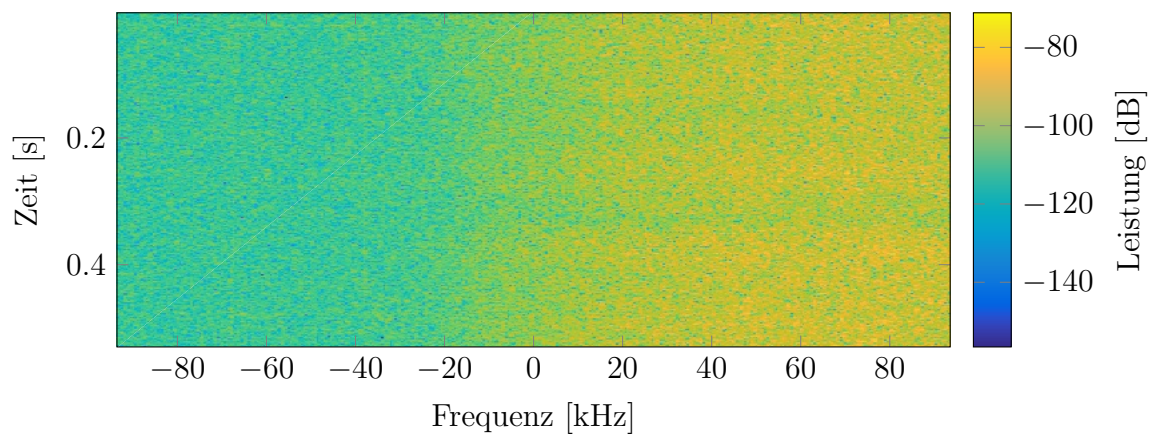
In diesem Abschnitt werden die konkreten Schritte zur Implementierung des Empfängers beschrieben. Der Fokus liegt dabei auf der praktischen Umsetzung der zuvor erläuterten theoretischen Konzepte in MATLAB. Für die Demodulation, Dekodierung und Bildrekonstruktion werden Funktionen der Communications Toolbox [29], der Signal Processing Toolbox [30] und der Image Processing Toolbox [31] verwendet.

Die Implementierung ist modular aufgebaut. Die Datei `main.m` führt die Verarbeitungskette aus: Konfiguration, Signalvoranalyse, Demodulation, Bestimmung der Konstellationsabbildung, Viterbi-Dekodierung, Rahmensynchronisation, Descrambling, Reed–Solomon-Korrektur, Paketextraktion und JPEG-Dekodierung. Die Funktion `getConfig` lädt die Aufnahme und die benötigten Parameter, darunter die Tabellen für die Huffman-Dekodierung und Dequantisierung.

Die untersuchte Aufnahme `meteor_m2_72k.wav` enthält 19 759 104 komplexe I/Q-Abtastwerte bei einer Abtastrate von 187 500 Hz. Dies entspricht einer Dauer von etwa 105,38 s. Die Symbolrate beträgt 72 000 Symbole pro Sekunde. Zur Voranalyse werden das Leistungsdichtespektrum und dessen zeitlicher Verlauf dargestellt. Abbildung 3.1 zeigt die Lage und Bandbreite des Empfangssignals sowie zeitliche Änderungen der spektralen Leistung.



(a) Leistungsdichtespektrum.



(b) Zeitlicher Verlauf des Spektrums.

Abbildung 3.1: Frequenzdarstellung des empfangenen Signals.

3.1 Demodulation

Die Demodulation erfolgt in zwei Funktionen. Zunächst werden die I/Q-Abtastwerte in Symbole überführt. Anschließend wird die verbleibende Mehrdeutigkeit der Symbol-Bit-Zuordnung aufgelöst, wie in Abschnitt 2.7 erläutert.

demod Die Funktion **demod** verarbeitet die I- und Q-Kanäle als **single**, begrenzt die Amplitude mittels **clip** und normiert beide Kanäle auf ihren jeweiligen Maximalbetrag. Ein Mindestwert im Nenner verhindert eine Division durch null bei stillen Abschnitten. Danach werden die Kanäle zum komplexen Basisbandsignal $x = I + jQ$ zusammengeführt.

Das Signal wird auf vier Abtastwerte pro Symbol, also 288 kHz, umgesetzt und mit einem Root-Raised-Cosine-Empfangsfilter gefiltert. Der Roll-off-Faktor beträgt 0,35, die Filterlänge umfasst zehn Symbolintervalle. Die weitere Verarbeitung erfolgt in Blöcken von 98 304 Abtastwerten: Grobe Frequenzkompensation, Taktrückgewinnung mit einem entscheidungsgeführten Zero-Crossing-Detektor und Trägersynchronisation liefern schließlich einen komplexen Abtastwert pro Symbol. Die Systemobjekte führen ihren Zustand zwischen den vollständigen Blöcken fort.

Auch ein kürzerer letzter Block wird verarbeitet, sofern er mindestens ein nominelles Symbolintervall enthält. Die Objekte für grobe Frequenzkompensation und Taktrückgewinnung werden hierfür an die geänderte Eingabegröße angepasst; dabei ist eine kurze erneute Einschwingphase möglich. Abbildung 3.2 veranschaulicht die Wirkung der

Demodulation anhand der Konstellation vor und nach der Synchronisation.

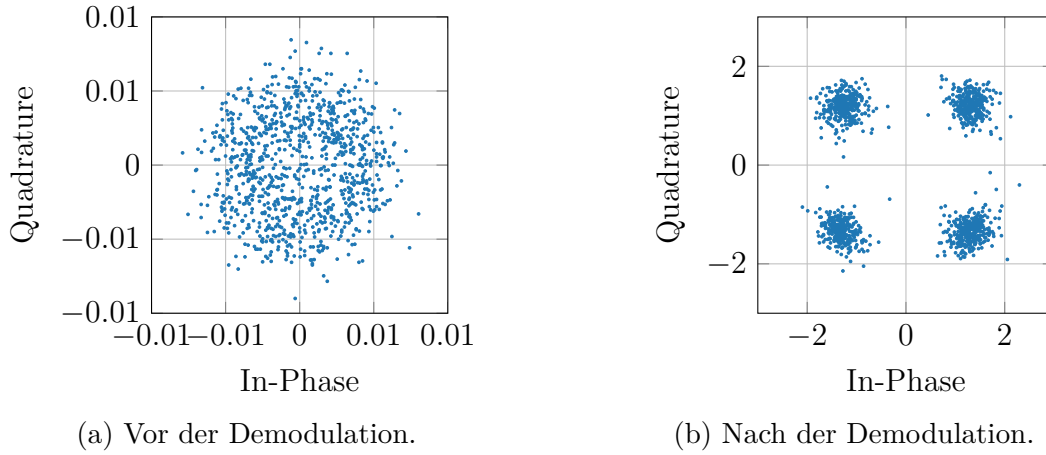


Abbildung 3.2: Konstellationsdiagramme des Signals vor und nach der Demodulation.

constellation Die Funktion `constellation` erzeugt aus den demodulierten Symbolen Softbits in Form von Log-Likelihood-Ratios (LLRs). Für jeden Symbolblock werden mehrere Konstellationsabbildungen geprüft, um verbleibende Rotationen und Spiegelungen aufzulösen. Als Referenz dient das faltungscodierte ASM `FCA2B63DB00D9794`, dessen Bits für die Korrelation auf $\{-1, +1\}$ abgebildet werden.

Eine CADU enthält vor der Faltungscodierung $1024 \cdot 8 = 8192$ Bits. Durch die Coderate $r = 1/2$ entstehen daraus 16 384 Kanalbits beziehungsweise LLR-Werte. Bei QPSK entsprechen diese 8192 Symbolen. Eine passende Konstellationsabbildung liefert daher Korrelationspeaks im Abstand von 16 384 LLR-Werten. Die Korrelationsamplitude wird normiert, und ein aus einem Quantil abgeleiteter Schwellenwert dient der Peakerkennung. Zusätzlich wird die Übereinstimmung der Peak-Abstände mit dem Frame-Raster geprüft. Die Polarität der akzeptierten LLRs wird konsistent zum Viterbi-Eingang gewählt.

Treten an Blockgrenzen unterschiedliche Abbildungen auf, werden die betreffenden Abschnitte getrennt und rekursiv ausgewertet. Die Rekursion ist begrenzt. Für Abschnitte ohne akzeptierte Zuordnung werden LLRs mit dem Wert null eingesetzt. Diese tragen keine Präferenz für einen der beiden Bitwerte und erhalten die Position des Abschnitts im Datenstrom. Abbildung 3.3 zeigt die Korrelation für einen synchronisierten Ausschnitt.

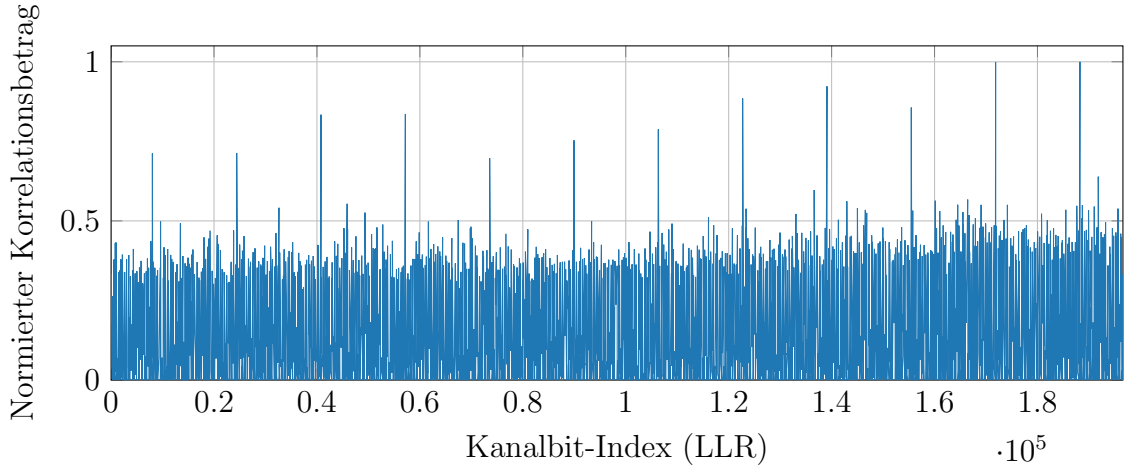


Abbildung 3.3: Normierter Betrag der Kreuzkorrelation des faltungscodierten ASM mit einem LLR-Block. Die Peaks liegen im Abstand von 16 384 Kanalbits.

3.2 Dekodierung

decode Die LLRs werden einem `comm.ViterbiDecoder` mit der Eingabeoption `Unquantized` zugeführt. Der Trellis wird für die Coderate $r = 1/2$, die Constraint Length $K = 7$ und die oktalen Generatorpolynome (171, 133) erzeugt. Die Tracebacktiefe beträgt 30. Positive LLRs stehen für Bit null, negative für Bit eins; der Betrag beschreibt die Zuverlässigkeit. Die Werte werden direkt übergeben, sodass ihre Vorzeichen und relativen Zuverlässigkeiten erhalten bleiben.

Zur Rahmensynchronisation werden die dekodierten Bits auf $\{-1, +1\}$ abgebildet und mit dem ASM `1ACFFC1D` korreliert. Der Betrag der Korrelation erreicht bei vollständiger Übereinstimmung mit dem 32-Bit-Marker den Wert 32. Die detektierten Peaks markieren mögliche Frame-Anfänge. Abbildung 3.4 zeigt einen Ausschnitt mit dem erwarteten Abstand von 8192 dekodierten Bits.

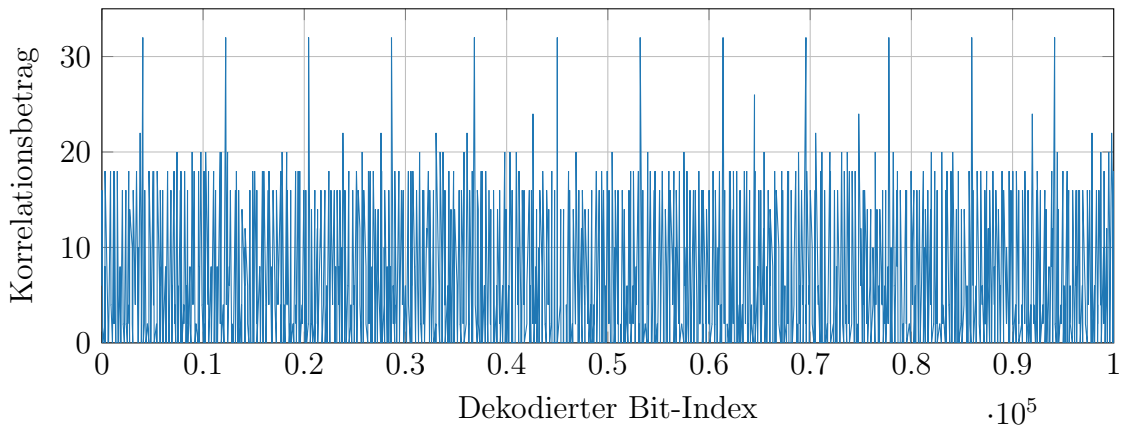


Abbildung 3.4: Betrag der Kreuzkorrelation des ASM 1ACFFC1D mit dem Viterbi-dekodierten Bitstrom.

Nach jedem Marker werden 8160 Bits als CVCDU entnommen. Ein letzter Frame wird berücksichtigt, wenn sein gesamtes Datenfeld vorliegt. Kürzere erkannte Abschnitte werden auf die feste Datenfeldlänge aufgefüllt; aus größeren Markerabständen abgeleitete fehlende Frames werden als Nullzeilen repräsentiert. Solche Platzhalter sind keine rekonstruierten Empfangsdaten und werden bei der Paketextraktion verworfen.

Für das Descrambling werden die extrahierten Bits in Bytes umgewandelt und elementweise mit der bekannten PN-Sequenz XOR-verknüpft. Die Sequenz beginnt bei jeder CVCDU erneut an derselben Position. Das ASM ist davon ausgenommen. Das Ergebnis wird als Array mit 1020 Bytes pro Frame abgelegt.

reedsolomon Nach dem Descrambling folgt die äußere Fehlerkorrektur. Jede CVCDU enthält vier verschachtelte Reed-Solomon-Codewörter mit je 255 Bytes. Der Code RS(255,223) besitzt 32 Prüfsymbole pro Codewort und kann bis zu 16 fehlerhafte Bytes pro Codewort korrigieren. Die vier Codewörter werden durch Entflechtung getrennt, einzeln über GF(256) dekodiert und wieder in die ursprüngliche Reihenfolge gebracht.

Zunächst werden die Syndrome berechnet. Bei einem von null verschiedenen Syndromvektor werden Fehlerpositionen und Fehlerwerte bestimmt und eine Korrektur versucht. Eine Korrektur wird nur übernommen, wenn die erneute Syndromprüfung null ergibt. Die Funktion liefert die bearbeiteten Frames und eine Fehlerstatistik pro Codewort. Nicht korrigierbare Codewörter werden mit einem negativen Status gemeldet; ihre ursprünglichen Bytes bleiben für die nachfolgenden Header- und Längenprüfungen erhalten.

Ein daraus gewonnenes Paket ist somit nicht allein durch seine plausible Struktur als fehlerfrei bestätigt.

extraction Die Funktion `extraction` entfernt die Reed–Solomon-Prüfbytes und zerlegt die VCDUs in ihre M-PDUs. Der FHP kennzeichnet den Beginn des ersten neuen Pakets innerhalb der 882 Byte langen Packet Zone. Bytes vor dieser Position können ein aus dem vorigen Frame fortgesetztes Paket vervollständigen. Der Pointerwert 2047 kennzeichnet einen Frame ohne neuen Paketanfang.

Unvollständige Paketheader und Nutzdaten werden in einem Puffer über Frame-Grenzen hinweg gesammelt. Dazu müssen die Frame-Zähler modulo 2^{24} unmittelbar aufeinanderfolgen und die Frame-Identifikatoren übereinstimmen. Bei einer Unterbrechung wird das bisherige Fragment verworfen. Die Paketlänge ergibt sich aus dem Längenfeld des Primärheaders zu $6 + (L + 1)$ Bytes. Headerfelder, APID, Qualitätsfaktormarker, Qualitätsfaktor und MCU-Position werden vor der Übernahme eines Bildpakets geprüft.

Abbildung 3.5 zeigt den vollständigen 24-Bit-Zähler der ausgewerteten Frames. Ein fortlaufender Zähler belegt die Kontinuität der Frame-Folge, ersetzt jedoch nicht die Prüfung der enthaltenen Daten.

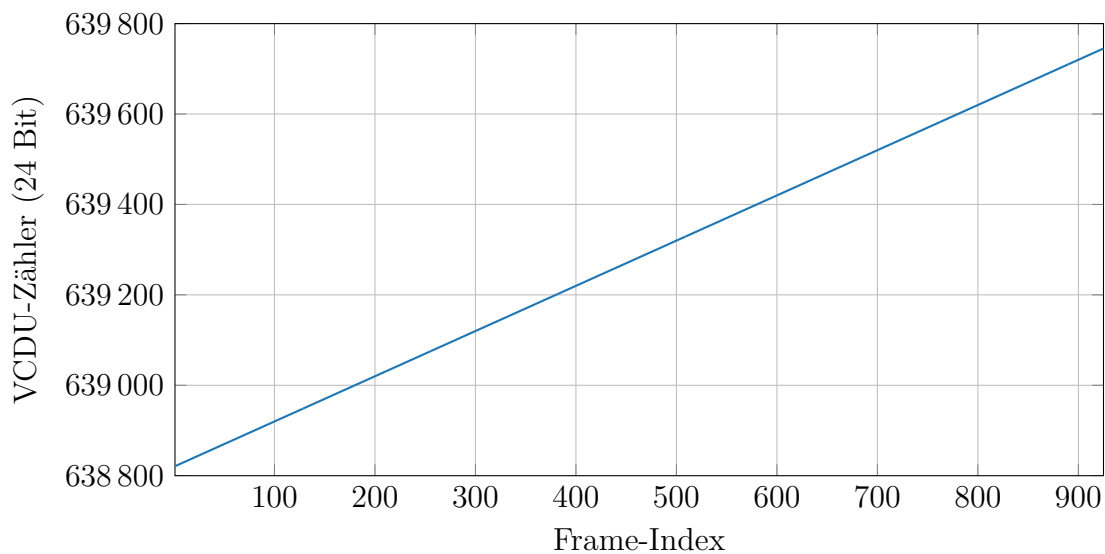


Abbildung 3.5: Vollständiger 24-Bit-VCDU-Zähler über die 925 ausgewerteten Frames.

Mit `keepPartialScans=true` werden auch gültige Bildpakete aus unvollständigen Bildstreifen beibehalten. Für jedes Paket werden der komprimierte Bitstrom, der Qualitätsfaktor, die APID, die Sequenznummer, der Zeitstempel und die Position der ersten MCU

gespeichert. Der Zeitstempel dient später zur gemeinsamen zeitlichen Zuordnung der Kanäle. Vollständig fehlende Zeitpunkte lassen sich aus den vorhandenen Paketen allein nicht rekonstruieren.

3.3 JPEG

huffman Die Funktion **huffman** erzeugt aus den in **getconfig** hinterlegten JPEG-Huffman-Tabellen die Codewort-Zuordnungen. Codewortlängen und Symbole werden gemäß der kanonischen Huffman-Konstruktion zu Bitmustern zusammengeführt. Bei den AC-Symbolen wird zusätzlich das Paar **run/size** ausgewertet. Die Tabellen entsprechen den Tabellen K.3 und K.5 des JPEG-Standards [25].

jpegdecoding In **jpegdecoding** werden die Bitströme der Bildpakete entropie- und laufhängendekodiert. Die Huffman-Dekodierung sucht ein gültiges Präfix im Bitstrom. Für den DC-Anteil gibt das gefundene Symbol die Länge s des folgenden Magnituden-Bitfeldes $b \in \{0,1\}^s$ an. Mit dessen dezimalem Wert $\text{val}(b)$ ergibt sich die vorzeichenbehaftete Magnitude zu

$$m = \begin{cases} 0, & s = 0, \\ \text{val}(b), & s > 0 \text{ und } b_1 = 1, \\ -((2^s - 1) - \text{val}(b)), & s > 0 \text{ und } b_1 = 0. \end{cases} \quad (3.1)$$

Für die AC-Koeffizienten beschreibt **run** die Anzahl vorangehender Nullen und **size** die Bitlänge des nächsten Nichtnullwerts. Das Symbol EOB (End of Block) setzt alle verbleibenden AC-Koeffizienten auf null. ZRL (Zero Run Length) steht für 16 aufeinanderfolgende Nullen. Die Dekodierung eines Blocks endet entweder mit EOB oder sobald sämtliche 63 AC-Positionen gefüllt sind. Die Bitfeldlängen und Koeffizientenpositionen werden vor jedem Zugriff überprüft. Ein nicht vollständig dekodierbarer Block erhält einen neutralen Platzhalter.

Die DC-Koeffizienten werden durch Aufsummieren ihrer Differenzen rekonstruiert. Der Prädiktor startet für das erste 8×8 -Teilbild jedes Bildpakets bei null. Die 64 Koeffizienten werden anschließend gemäß der Zig-Zag-Reihenfolge in eine 8×8 -Matrix einsortiert.

Der Qualitätsfaktor q_i bestimmt für jedes Bildpaket den Skalierungsfaktor der Basistabelle.

Es gilt

$$F(q_i) = \begin{cases} 5000/q_i, & 20 < q_i < 50, \\ 200 - 2q_i, & 50 \leq q_i \leq 100, \\ 100, & \text{sonst.} \end{cases} \quad (3.2)$$

Die Einträge der tatsächlich verwendeten Quantisierungstabelle sind ganzzahlig und mindestens eins:

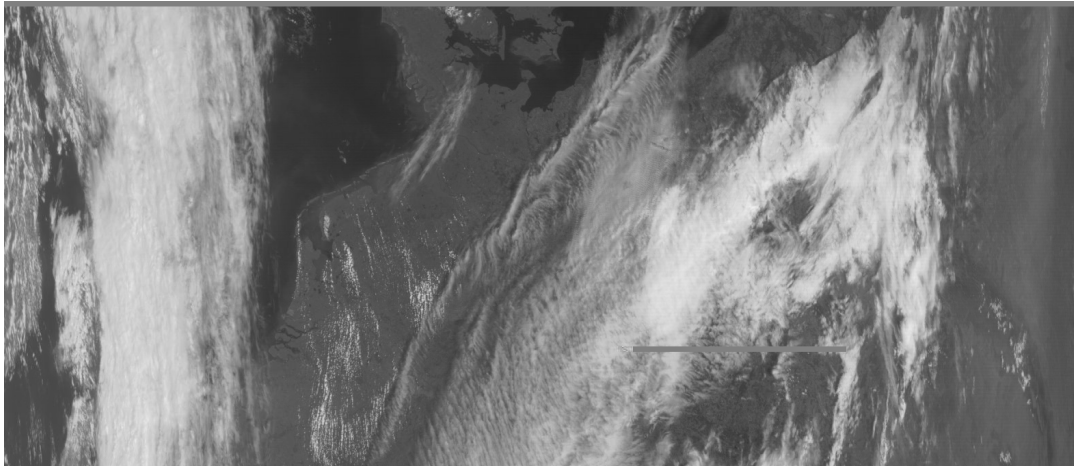
$$Q_{u,v}(q_i) = \max\left(1, \left\lfloor \frac{Q_{u,v}^{\text{Basis}} F(q_i) + 50}{100} \right\rfloor\right). \quad (3.3)$$

Damit besteht die Tabelle für $q_i = 100$ aus Einsen. Die dequantisierten Koeffizienten entstehen durch elementweise Multiplikation mit $Q(q_i)$. Die Funktion `idct2` liefert daraus den Bildblock im Ortsraum. Nach Addition des Level-Shifts von 128 erfolgt die Umwandlung in den Wertebereich eines 8-Bit-Grauwertbildes.

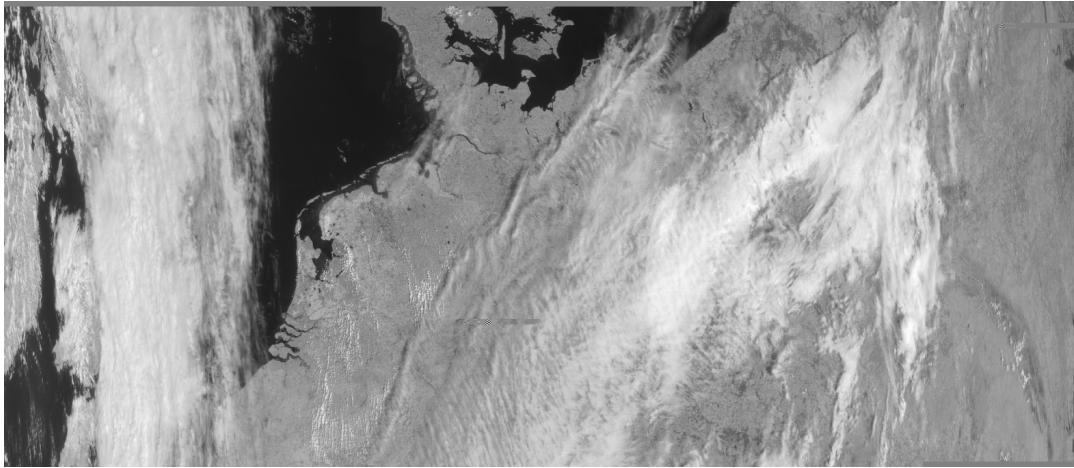
Jedes Bildpaket enthält 14 MCUs zu je 8×8 Pixeln. Ein vollständiger Streifen besteht aus 14 solchen Paketen und hat die Abmessungen 1568×8 Pixel. Die Paketpositionen innerhalb eines Streifens werden durch die MCU-Startnummern $0, 14, \dots, 182$ beschrieben. Die Funktion ordnet jeden Block anhand dieser Position und des Paketzeitstempels dem jeweiligen Kanal zu. Alle drei Kanäle verwenden dieselbe sortierte Liste der vorhandenen Zeitstempel. Dadurch beziehen sich gleich positionierte Bildstreifen auf denselben Aufnahmezeitpunkt, auch wenn in einem Kanal Pakete fehlen.

Fehlende oder nicht dekodierbare Blöcke erhalten den Grauwert 128. Die Ausgaben `valid64`, `valid65` und `valid68` markieren, welche Bereiche tatsächlich dekodiert wurden; sie sind kein Nachweis der radiometrischen oder vollständigen Übertragungsfehlerfreiheit. Während der Dekodierung werden die drei Kanalbilder nach jedem Paket aktualisiert. Die Darstellung zeigt damit den fortschreitenden Aufbau an den jeweiligen Bildpositionen.

Das RGB-Komposit verwendet die Zuordnung $R = I_{64}$, $G = I_{65}$ und $B = 255 - I_{68}$. Es kombiniert zwei Kanäle direkt und den invertierten IR-Kanal als Blauanteil. Das Ergebnis ist ein Falschfarbenkomposit und kein kalibriertes Echtfarbenbild. Abbildung 3.6 zeigt die drei rekonstruierten Kanäle und das Komposit ohne zusätzliche Schärfung, Entrauschung oder Kontrastanpassung.

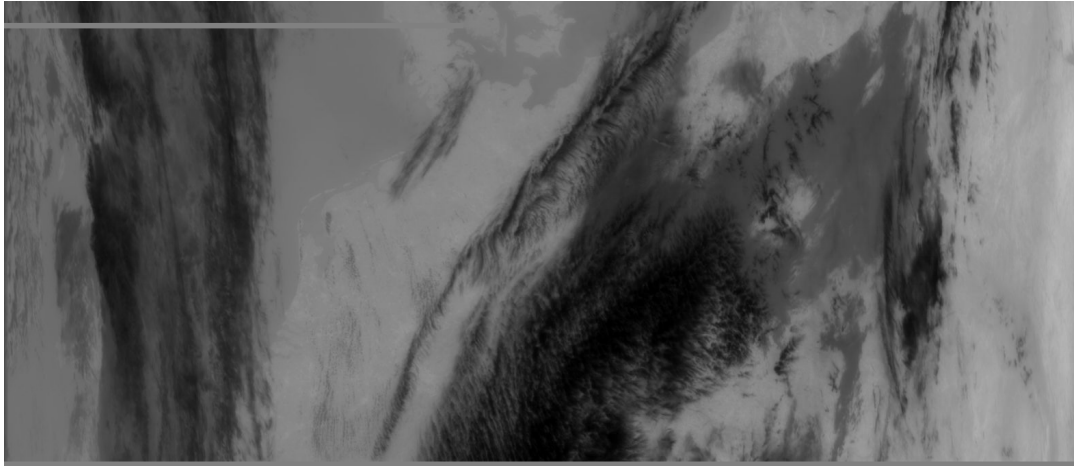


(a) APID 64.

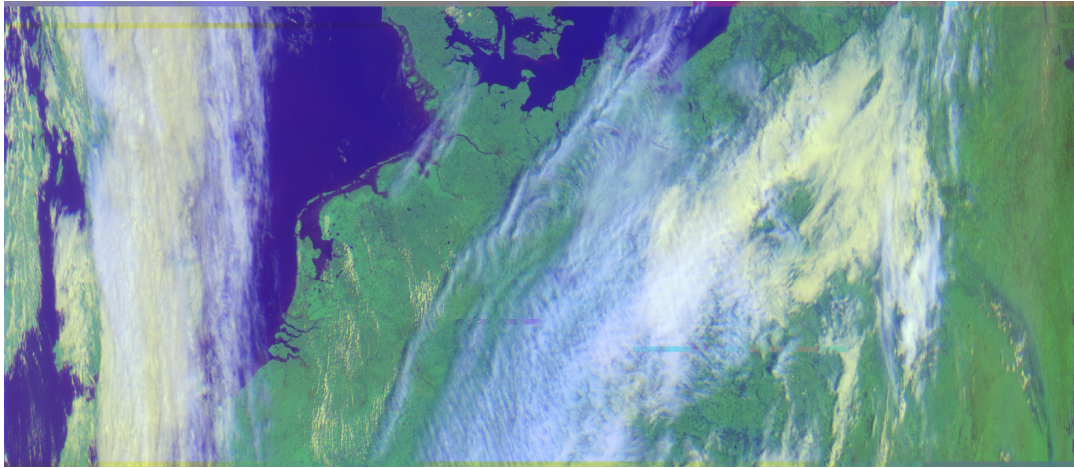


(b) APID 65.

Abbildung 3.6: Dekodierte Bildkanäle und RGB-Komposit mit jeweils 1568×680 Pixeln. Graue Platzhalter in den Einzelkanälen kennzeichnen nicht rekonstruierte Bildblöcke.



(c) APID 68.



(d) Falschfarbenkomposit: $R = I_{64}$, $G = I_{65}$, $B = 255 - I_{68}$.

Abbildung 3.6: Dekodierte Bildkanäle und RGB-Komposit (Fortsetzung).

3.4 Ergebnisse und Validierung

Die Aufnahme wurde mit MATLAB R2026b ausgewertet. Tabelle 3.1 fasst die Ergebnisse zusammen. Aus 925 Frames werden 3523 Bildpakete gewonnen. Die gemeinsame Zeitachse umfasst 85 Bildstreifen zu jeweils acht Pixelzeilen und ergibt eine Bildhöhe von 680 Pixeln. Die Qualitätsfaktoren der Bildpakete liegen zwischen 71 und 100.

Tabelle 3.1: Ergebnisse für die untersuchte I/Q-Aufnahme.

Größe	Ergebnis
Ausgewertete Frames	925
Rekonstruierte Bildpakete	3523
Gemeinsame Bildstreifen	85
Bildabmessungen je Kanal	1568×680 Pixel
Korrigierte RS-Symbole (Bytes)	25
RS-gültige bzw. korrigierte Frames	921 von 925
Nicht korrigierbare RS-Codewörter	16 in vier Frames

Die Funktionsprüfung umfasst 14 automatisierte Tests mit bekannten Referenzdaten. Geprüft werden unter anderem Viterbi-Dekodierung, vollständige und verkürzte letzte Eingabeblocke, Stille und Rauschen, Reed–Solomon-Korrektur, JPEG-Blockgrenzen, über mehrere Frames verteilte Pakete sowie die Platzierung bei fehlenden Kanaldaten. Für Reed–Solomon werden Fehlerzahlen von null bis 16 je Codewort geprüft; ein gezielt gewählter Testfall mit 17 Fehlern wird als nicht korrigierbar erkannt. Daraus folgt keine allgemeine Erkennungsgarantie für beliebige Fehlermuster außerhalb des Korrekturradius.

Zusätzlich wurden 84 Bildpakete aus der Aufnahme über alle vorkommenden Qualitätsfaktoren und die drei verwendeten Kanäle mit einer unabhängigen Pillow/libjpeg-Dekodierung verglichen. Die mittlere absolute Abweichung beträgt 0,0151 Grauwerte, die maximale Abweichung einen Grauwert. Die geringe Differenz ist mit unterschiedlichen Rundungen der inversen DCT vereinbar. Dieser Vergleich prüft die JPEG-Rekonstruktion der empfangenen Pakete; ohne ein fehlerfreies Originalbild lässt sich daraus keine absolute Bildgüte der gesamten Empfangskette ableiten.

4 Fazit

In diesem Projekt wurde eine durchgängige Verarbeitungskette für eine reale I/Q-Aufnahme des Meteor-M2-Satelliten implementiert. Sie verbindet Frequenz- und Taktsynchronisation, QPSK-Demodulation, Viterbi-Dekodierung, Rahmensynchronisation, Descrambling und Reed–Solomon-Korrektur mit der paketweisen Rekonstruktion der JPEG-Bilddaten. Damit werden die theoretischen Konzepte der digitalen Kommunikation an einem vollständigen Empfangsprozess praktisch nachvollziehbar.

Die untersuchte Aufnahme liefert 925 Frames und 3523 Bildpakete. Aus diesen entstehen drei zeitlich zugeordnete Kanalbilder sowie ein Falschfarbenkomposit mit jeweils 1568×680 Pixeln. Die Reed–Solomon-Dekodierung korrigiert 25 Symbolfehler; vier Frames enthalten weiterhin nicht korrigierbare Codewörter. Fehlende oder nicht dekodierbare Bildblöcke werden durch Platzhalter und Gültigkeitsmasken kenntlich gemacht. Die Auswertung beschreibt daher sowohl die rekonstruierte Bildinformation als auch die verbleibenden Datenlücken.

Für die Bildrekonstruktion sind die ganzzahlige Dequantisierung, die korrekte Behandlung der JPEG-Blockgrenzen und die gemeinsame zeitliche Zuordnung der Kanäle wesentlich. Bekannte synthetische Referenzen prüfen die einzelnen Verarbeitungsschritte. Der zusätzliche Vergleich echter Bildpakete mit einer unabhängigen JPEG-Implementierung bestätigt die numerische Übereinstimmung der Rekonstruktion bis auf geringe Rundungsabweichungen. Eine absolute Bewertung der gesamten Bildqualität bleibt ohne ein fehlerfreies Referenzbild begrenzt.

Weiterführende Arbeiten können die Fehlerstatistik über mehrere Aufnahmen und unterschiedliche Empfangsbedingungen untersuchen. Auch eine zustandserhaltende Verarbeitung beliebiger letzter Blocklängen, die Weitergabe des RS-Status bis zur einzelnen Bildregion sowie eine radiometrische Kalibrierung und geografische Zuordnung der Kanäle bieten sich an. Hinsichtlich der Laufzeit bestehen weitere Möglichkeiten zur Vektorisierung und zur effizienteren Verwaltung der Daten. Die modulare MATLAB-Implementierung bildet dafür eine nachvollziehbare Grundlage.

Literatur

- [1] G. K. Davis, *History of the NOAA satellite program*, *Journal of Applied Remote Sensing*, Jg. 1, Nr. 1, S. 012 504, Jan. 2007. DOI: 10.1117/1.2642347
 - [2] P. V. Dascal, P. Dolea, O. Cristea und T. Palade, *LRPT Weather Satellite Image Acquisition Using a SDR-Based Reception System*, *Acta Technica Napocensis Electronics and Telecommunications*, Jg. 56, Nr. 2, Juni 2015.
 - [3] K. Singh und A. Nirmal, *Overview of modulation schemes selection in satellite based communication*, *ICTACT J. Commun. Technol*, Jg. 11, Nr. 03, S. 2203–2207, Sep. 2020. DOI: 10.21917/ijct.2020.0326
 - [4] R. A. Stampfl und W. G. Stroud, *Automatic Picture Transmission TV Camera System for Meteorological Satellites*, *Journal of the SMPTE*, Jg. 73, Nr. 2, S. 130–134, 1964.
 - [5] W. Fong, P.-S. Yeh, S. Duran, V. Sank, X. Nyugen, W. Xia und J. H. Day, *Low Resolution Picture Transmission (LRPT) Demonstration System*, National Aeronautics und Space Administration, Goddard Space Flight Center, Techn. Ber., 2002.
 - [6] P. Noll und T. Sikora, „Digitale Nachrichtenübertragung,“ (Vorlesungsskript), 2023.
 - [7] J. P. Costas, *Synchronous Communications*, *Proceedings of the IRE*, Jg. 44, Nr. 12, S. 1713–1718, 1956. DOI: 10.1109/JRPROC.1956.275063
 - [8] M. Luise und R. Reggiannini, *Carrier frequency recovery in all-digital modems for burst-mode transmissions*, *IEEE Transactions on Communications*, Jg. 43, Nr. 2/3/4, S. 1169–1178, Feb. 1995. DOI: 10.1109/26.380149
 - [9] F. Gardner, *Phaselock Techniques: Third Edition*. Hoboken, NJ: John Wiley & Sons, Inc., Juli 2005, S. 1–425. DOI: 10.1002/0471732699
 - [10] Q. Chaudhari, *Wireless Communications from the Ground Up: An SDR Perspective*. Melbourne, Australia: Self-published, 2018.
 - [11] I. N. Bronshtein, K. A. Semendyayev, G. Musiol und H. Mühlig, *Taschenbuch der Mathematik*, 7. Aufl. Berlin, Heidelberg: Springer, 2013. DOI: 10.1007/978-3-8348-2359-5
-

-
- [12] Y. Wang, K. Shi und E. Serpedin, *Non-Data-Aided Feedforward Carrier Frequency Offset Estimators for QAM Constellations: A Nonlinear Least-Squares Approach*, *EURASIP Journal on Applied Signal Processing*, Jg. 2004, Nr. 15, S. 2405–2414, Okt. 2004. DOI: 10.1155/S1110865704403175
- [13] K. Mueller und M. Muller, *Timing Recovery in Digital Synchronous Data Receivers*, *IEEE Transactions on Communications*, Jg. 24, Nr. 5, S. 516–531, Mai 1976. DOI: 10.1109/TCOM.1976.1093326
- [14] S. Chen und A. M. Wyglinski, *Chapter 3 - Digital communication fundamentals for cognitive radio*. Oxford: Academic Press, 2010, S. 41–83. DOI: 10.1016/B978-0-12-374715-0.00003-4
- [15] U. Mengali und A. N. D’Andrea, *Synchronization Techniques for Digital Receivers*. New York, NY: Springer Science+Business Media, LLC, 1997. DOI: 10.1007/978-1-4615-6167-1
- [16] M. Rice, *Digital Communications: A Discrete-Time Approach*. Upper Saddle River, NJ: Prentice Hall, 2008.
- [17] J. G. Proakis und M. Salehi, *Digital Communications*, 5. Aufl. New York, NY: McGraw-Hill, 2007.
- [18] C. E. Shannon, *A mathematical theory of communication*, *The Bell System Technical Journal*, Jg. 27, Nr. 3, S. 379–423, Juli 1948. DOI: 10.1002/j.1538-7305.1948.tb01338.x
- [19] H. Imai, Hrsg., *Essentials of Error-Control Coding Techniques*. San Diego, CA: Academic Press, 1990. DOI: 10.1016/C2013-0-10897-5
- [20] Consultative Committee for Space Data Systems, *TM Synchronization and Channel Coding*, CCSDS, Blue Book CCSDS 131.0-B-5, Sep. 2023.
- [21] A. Viterbi, *Error bounds for convolutional codes and an asymptotically optimum decoding algorithm*, *IEEE Transactions on Information Theory*, Jg. 13, Nr. 2, S. 260–269, Apr. 1967. DOI: 10.1109/TIT.1967.1054010
- [22] J.-M. Friedt, *Meteor-M 2 Low Resolution Picture Transmission (LRPT) Decoding and Image Reconstruction*, Université de Franche-Comté/FEMTO-ST Institute, Techn. Ber., 2019.
-

-
- [23] A. GmbH, EUMETSAT und ESA, *MetOp Space to Ground Interface Control Document (ICD)*, European Organisation for the Exploitation of Meteorological Satellites (EUMETSAT), Darmstadt, Germany, Techn. Ber. MO-IF-MMT-SY0001, Version Issue 07, Revision C, März 2004.
- [24] Russische Raumfahrtssysteme, *Bortovaya informatsionnaya sistema s radioliniyey metrovogo diapazona (BIS-MV) KA MMeteor-MNNo. 2: Struktura informatsionnogo potoka*, AO Rossiyskiye kosmicheskiye sistemy, Moscow, Russia, Techn. Ber., 2014.
- [25] International Telecommunication Union, *Information Technology - Digital Compression and Coding of Continuous-Tone Still Images - Requirements and Guidelines*, ITU-T, Geneva, Switzerland, Techn. Ber. Recommendation T.81, 1992.
- [26] G. Wallace, *The JPEG still picture compression standard*, *IEEE Transactions on Consumer Electronics*, Jg. 38, Nr. 1, S. xviii–xxxiv, Feb. 1992. DOI: 10.1109/30.125072
- [27] D. A. Huffman, *A method for the construction of minimum-redundancy codes*, *Proceedings of the IRE*, Jg. 40, Nr. 9, S. 1098–1101, Sep. 2007. DOI: 10.1109/JRPROC.1952.273898
- [28] K. R. Rao und P. Yip, *Discrete cosine transform: algorithms, advantages, applications*. San Diego, CA: Academic press, 2014.
- [29] The MathWorks, Inc., *Communications Toolbox*, Version R2026b, Natick, Massachusetts, United States, 2026.
- [30] The MathWorks, Inc., *Signal Processing Toolbox*, Version R2026b, Natick, Massachusetts, United States, 2026.
- [31] The MathWorks, Inc., *Image Processing Toolbox*, Version R2026b, Natick, Massachusetts, United States, 2026.
-